

Relatório Técnico do Projeto da Unidade Curricular: Fundamentos de Desenvolvimento de Software

Diogo Ribeiro 1250871, João Bessa 1251089, Manuel Torre 1251209,
Shantall Camacho 1251460

Grupo 5, Turma 1DA

Resumo: Este projeto consistiu no desenvolvimento de um Sistema de Gestão de Planos de Viagem (SGPV) em C++, desenhado sob a arquitetura MVC (Model-View-Controller) para garantir uma separação rigorosa entre a lógica de negócio, a interface de terminal e o controlo de fluxos. Através da implementação de uma estrutura de herança e polimorfismo (utilizando a classe base Localidade para gerir Pais, Cidade e Atividade), o sistema permite ao utilizador registar-se, explorar um catálogo dinâmico carregado via ficheiros externos e construir roteiros personalizados com validação automática de calendários e cronogramas. A robustez da aplicação é sustentada pela aplicação prática dos princípios SOLID e padrões GRASP, que asseguram alta coesão e baixo acoplamento, como evidenciado na criação de classes de "Fabricação Pura" para cálculos financeiros e na gestão especializada de exceções para garantir a integridade dos dados. O resultado final é um software profissional e escalável que automatiza desde o planeamento logístico até à emissão de faturas detalhadas com aplicação de taxas de IVA, demonstrando um domínio avançado das boas práticas de Engenharia de Software e da linguagem C++.

Palavras-chave: C++ (POO), Arquitetura MVC, Princípios SOLID, Padrões GRASP, Polimorfismo, Herança, Gestão de Itinerários, Validação de Dados, Persistência de Ficheiros, Tratamento de Exceções.

1. Introdução

1.1 A Digitalização do Ecossistema Turístico

No paradigma atual da Sociedade, o setor do turismo desde, há alguns anos, tem vindo a atravessar uma metamorfose digital o que tem levado a surgir certas necessidades para priorizar a sua sobrevivência. A transição dos métodos convencionais de reserva para sistemas de planeamento autónomos não passa apenas por uma questão de conveniência, mas já por uma exigência de eficiência operacional e integridade de dados. O planeamento de uma jornada contemporânea envolve a intersecção de múltiplas variáveis: restrições temporais, flutuações orçamentais, requisitos logísticos de transporte e a gestão dos perfis dos respetivos utilizadores.

Neste cenário, surge a necessidade de ferramentas que não se limitem ao armazenamento, mas que atuem como sistemas de apoio à decisão, permitindo que o organizador simule e valide roteiros complexos antes da sua execução.

1.2 A Problemática e a Relevância do Software

O desenvolvimento de um Sistema de Gestão e Planeamento de Viagens (SGPV) enfrenta desafios clássicos da Engenharia de Software. A principal problemática reside na modelação da complexidade: como representar, de forma coerente e escalável, a relação entre entidades geográficas estáticas (Países e Cidades), entidades lógicas dinâmicas (Roteiros, Agendas e Faturas) e entidades biográficas (organizador, viajante)?

A ausência de um sistema integrado resulta frequentemente em redundância de dados e erros de cálculo financeiro, associação ou contagem. Assim, este projeto justifica-se pela procura de uma solução que aplique o rigor da Programação Orientada a Objetos (POO) para mitigar estas falhas, garantindo que cada componente do sistema tenha uma responsabilidade única e bem definida.

1.3 Objetivos e Âmbito do Projeto

O objetivo central deste trabalho é o desenvolvimento de uma aplicação robusta em C++ que sirva como plataforma integral para o ciclo de vida de uma viagem. Os objetivos específicos enumeram-se em:

- **Abstração e Modelação:** Criar uma hierarquia de classes (através de herança e polimorfismo) que represente fielmente a estrutura de um catálogo global.
- **Gestão de Fluxo Lógico:** Implementar uma camada de controlo composta por 15 controladores especializados, garantindo que a lógica de negócio (ex: validação de datas, cálculo de IVA) está isolada da interface de utilizador.

- Persistência Heterogénea: Desenvolver mecanismos de serialização para leitura de bases de dados em texto (.txt) e escrita de ficheiros de histórico e dados de viagens, assegurando a continuidade da informação.
- Robustez Sistémica: Garantir a resiliência da aplicação através de um sistema centralizado de gestão de exceções.

1.3.1 Classificação Tripartida de Entidades

Para que o SGPV opere com integridade, a arquitetura de dados foi desenhada em torno de três eixos fundamentais:

- Entidades Geográficas (Estáticas): Representam o referencial físico do sistema, sendo, por exemplo, as classes Pais e Cidade. São consideradas estáticas pois a sua existência precede a execução do programa e os seus atributos, como seriam o nome e o ID, raramente sofrem mutações durante o ciclo de vida de uma viagem.
- Entidades de Ator ou Biográficas (Sujeitos): Aqui enquadram-se as classes Organizador e Viajante. Estas entidades representam os sujeitos da ação. Ao contrário de um destino, estas entidades possuem atributos biográficos, como nome, idade, credenciais de acesso, entre outros, e são os pivôs em torno dos quais a viagem é construída. No caso do Viajante, a sua modelação é crítica para a lógica de negócio, uma vez que atributos como a idade podem ditar regras de validação, como poderiam ser a incapacidade de entrar num certo grupo grupo que se pretende fazer um certa viagem, devido à falta de uma idade mínima necessária.
- Entidades Lógicas e Transacionais (Dinâmicas): São as entidades de processo, como Viagem, Agenda e Fatura. Elas não existem de forma isolada; resultam da intersecção entre um Sujeito, quem viaja, e um referencial geográfico, para onde viaja. São dinâmicas porque são geradas, modificadas e liquidadas conforme o utilizador interage com o sistema.

1.4 Metodologia de Desenvolvimento e Arquitetura

A metodologia adotada baseia-se no princípio da Modularidade. Utilizou-se a linguagem C++ pela sua eficiência na gestão de memória e pelo suporte nativo a tipos de dados abstratos. A arquitetura foi desenhada seguindo o padrão MVC (Model-View-Controller), com uma ênfase particular nos princípios GRASP (*General Responsibility Assignment Software Patterns*).

O nosso trabalho visa reduzir o acoplamento(coupling) entre as funcionalidades de checkout, planeamento e catálogo, facilitando futuras expansões do sistema, como a integração com APIs externas ou interfaces gráficas.

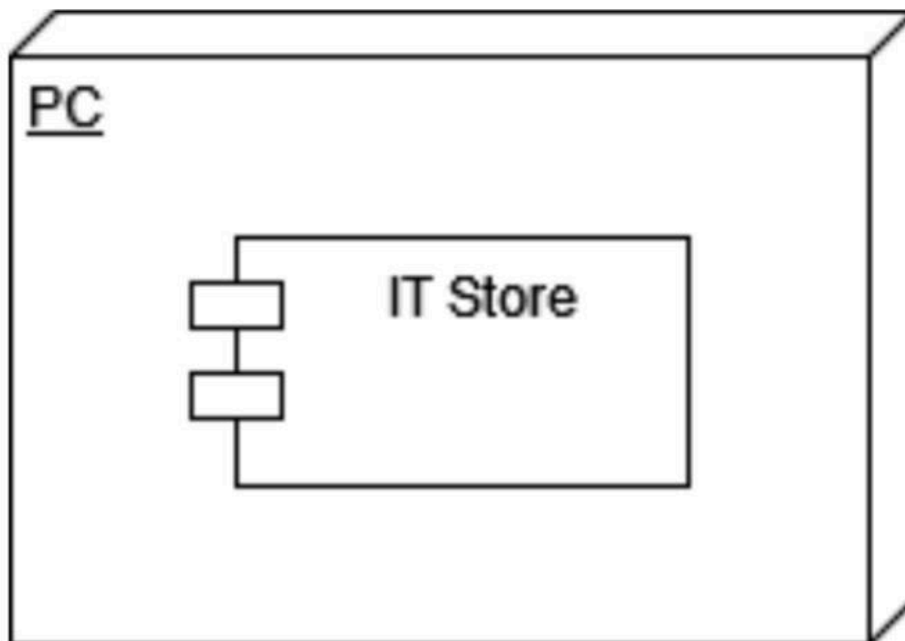
2. Arquitetura e Design do Sistema

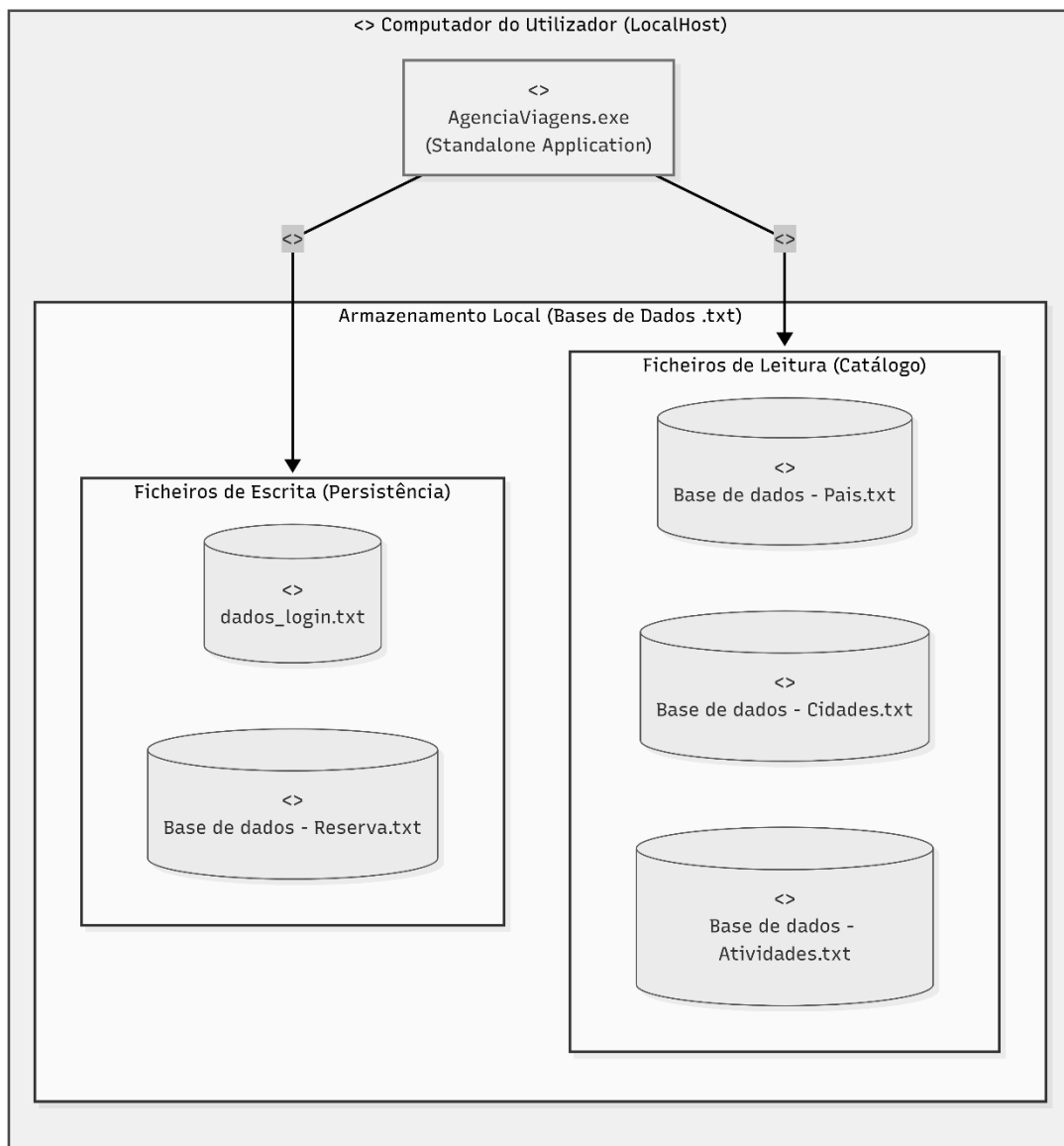
A conceção do SGPV foi orientada pelos princípios da robustez e da separação de interesses. A arquitetura não se limita a organizar o código, mas define como a informação flui e como o sistema se comporta perante falhas.

2.1 Diagrama de Implantação: Arquitetura física

O sistema adota uma topologia de Software como Produto (Standalone). Ao contrário de sistemas em nuvem, o SGPV é auto-suficiente no nó de execução.

- **Processamento Local (Compute Node):** O hardware aloja o ciclo de vida completo do processo. A execução é síncrona, garantindo que as operações de escrita em ficheiro não sofram latência de rede.
- **Gestão de Artefactos:** O binário compilado interage com o sistema de ficheiros através de fluxos de I/O (fstream). Esta escolha garante a portabilidade: o sistema pode ser movido entre diferentes diretórios sem perder a integridade, desde que a estrutura de caminhos (*paths*) para os ficheiros .txt seja mantida.
- **Ciclo de Persistência:** Diferenciamos o armazenamento de Configuração, o catálogo imutável do armazenamento de Sessão, o que consideramos como histórico de viagens mutável. Esta separação lógica no diagrama de implantação vai prevenir a corrupção de dados durante a gravação de ficheiros de utilizador.





2.2 Diagrama de Componentes: Arquitetura lógica

A implementação do padrão MVC no SGPV não é rígida, mas sim adaptada para uma aplicação de consola de alta complexidade.

1. O Subsistema de Controlo:

- Granularidade: Dividimos a lógica em 15 controladores para evitar o antipadrão God Object, um único objeto que faz tudo. Cada controlador, funciona como um perito de domínio.
- Orquestração: Os controladores comunicam entre si através de trocas de mensagens e referências. Por exemplo, o CheckoutController solicita dados ao GestaoCarrinhoController, processa-os e invoca o

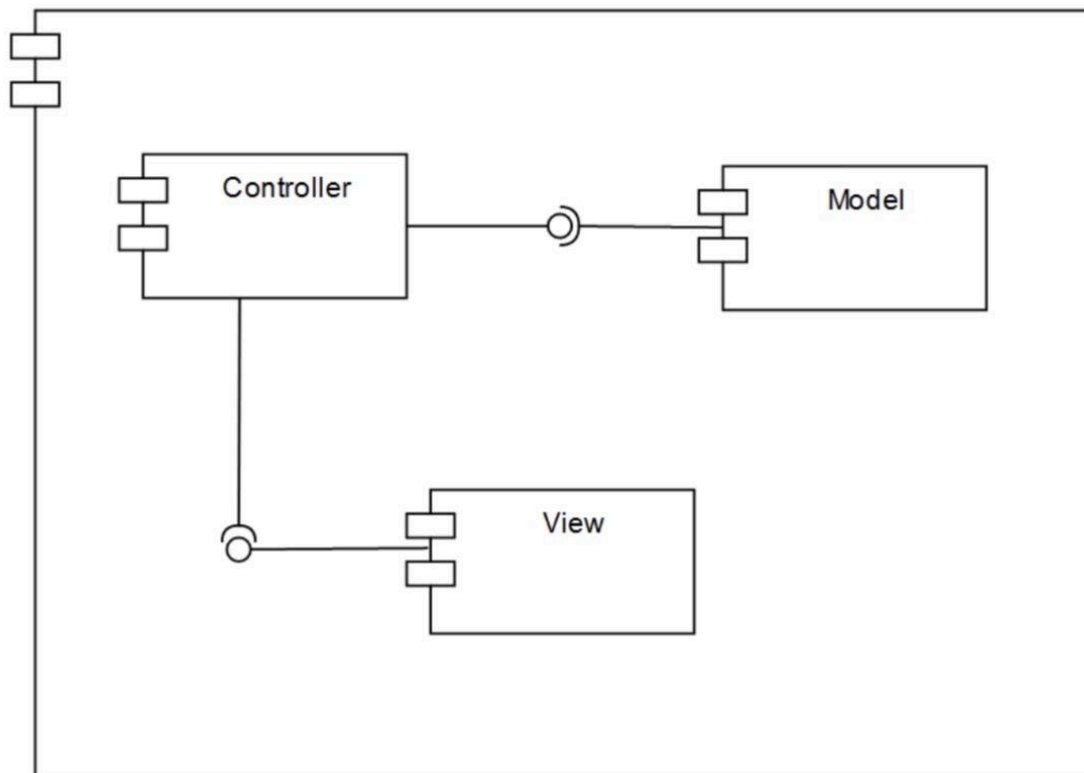
FaturaController. Esta "passagem de testemunho" assegura que nenhuma classe detém demasiada responsabilidade.

2. O Domínio de Dados:

- Baseia-se numa Hierarquia de Herança. A classe abstrata Localidade define a interface comum, que seriam o obterID(), obterPreco(), enquanto Pais e Cidade implementam as especializações. Isto permite que o CatalogoController trate qualquer destino de forma polimórfica.

3. Cross-Cutting Concerns:

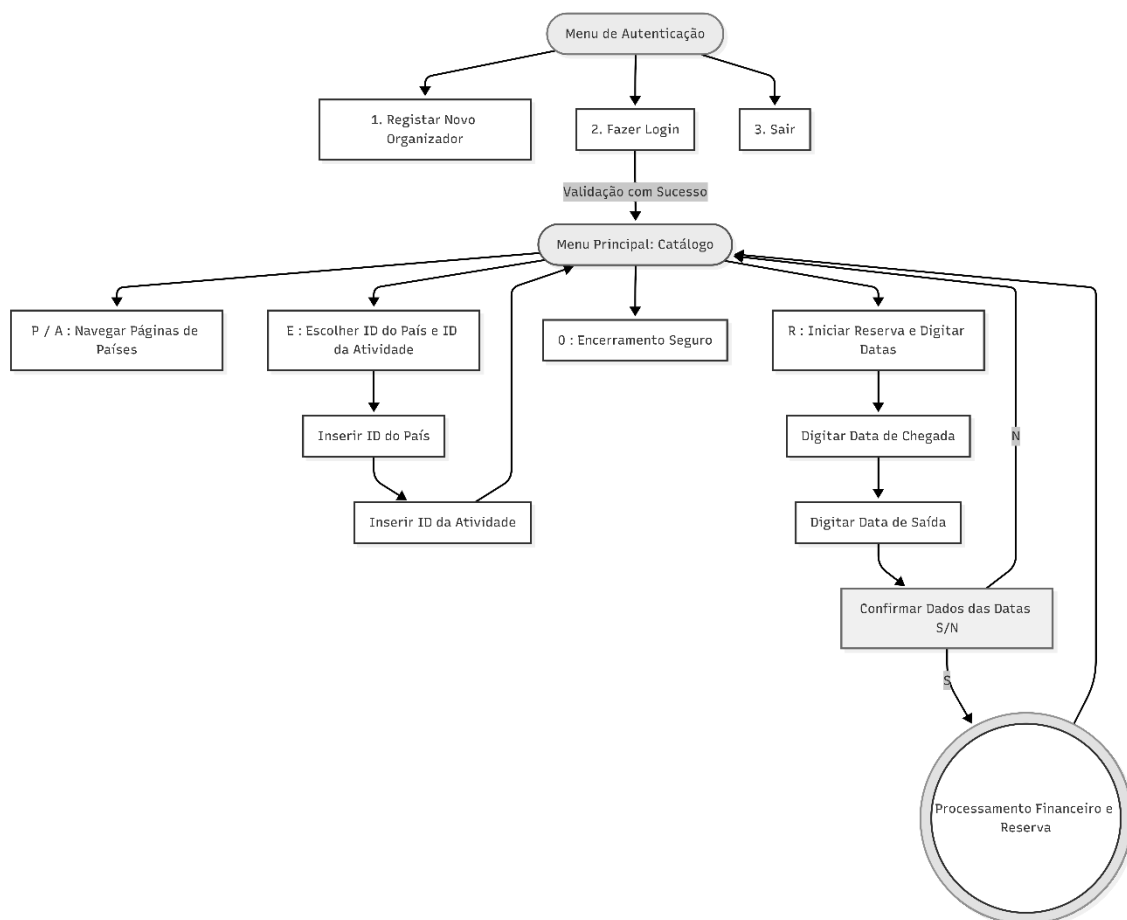
- O LoggerController é um componente transversal. Ele não pertence a uma camada específica; ele "observa" todas. Qualquer falha na camada de Persistência, como seria um ficheiro corrompido, ou na Model, datas inválidas, como seriam entidades como o organizador, é centralizada aqui, garantindo um único ponto de auditoria de erros.



2.3 Árvore de UI: Design de Interação e Máquina de Estados

A Interface de Utilizador (UI) foi modelada como uma Árvore de Estados Hierárquica. Cada nó da árvore representa um estado do sistema onde apenas certas operações são permitidas.

- Estado de Pré-Authenticação: O sistema inicia num estado restrito. A transição para o "Menu Principal" só ocorre após o sinal positivo do LoginController.
- Contexto de Planeamento (Wizard Design Pattern): O planeamento de uma viagem foi desenhado como um fluxo linear (Título → Datas → Destinos). Esta escolha de UI impede a criação de "viagens órfãs" (viagens sem destinos ou sem datas), forçando a integridade dos dados logo na origem do *input*.
- Ramificação de Gestão (Histórico e Edição): Ao seleccionar uma viagem no histórico, o sistema entra num modo de Edição Transaccional. Através do EditorViagemController, o utilizador pode modificar o estado da entidade Viagem em memória antes de confirmar a persistência definitiva no ficheiro, funcionando quase como um sistema de "Commit" de base de dados.



3. Diagrama de classes

O Diagrama de Classes apresentado constitui a espinha dorsal estrutural do Sistema de Gestão de Planos de Viagem (SGPV). Este modelo descreve a organização estática da aplicação, detalhando as entidades, os seus atributos, métodos e, fundamentalmente, as

relações de hierarquia e associação que permitem a persistência e manipulação dos dados. A nossa arquitetura baseia-se em quatro pilares principais, visíveis na organização do diagrama.

3.1 Núcleo Logístico e de Gestão (Viagem e Roteiro)

O centro do sistema é a classe Viagem, que atua como o agregador principal de toda a informação. Ela possui uma relação de composição com o Roteiro, que por sua vez gere uma coleção de objetos do tipo Localidade. Esta estrutura permite que uma viagem seja composta por múltiplos destinos e atividades de forma dinâmica.

3.2 Hierarquia de Dados do Catálogo

Um ponto de destaque no projeto é a classe abstrata Localidade. Dela derivam as classes País, Cidade e Atividade.

- Esta hierarquia utiliza o conceito de herança, onde as classes filhas herdam atributos base (ID, Nome), mas implementam o seu próprio comportamento no método virtual obterPreco().
- A classe RepositorioCatalogo é a responsável pela gestão destas coleções, servindo de interface entre o armazenamento em disco e a memória RAM.

3.3 Gestão de Utilizadores e Autenticação

A estrutura de utilizadores é definida através da classe base Pessoa, que se especializa em Organizador (o responsável pelo plano) e Viajante. A classe Login assegura a segurança e persistência das credenciais, mantendo uma associação com o Organizador para vincular cada plano ao seu criador.

3.4 Componentes Transversais

O sistema conta com módulos utilitários que garantem o rigor da aplicação:

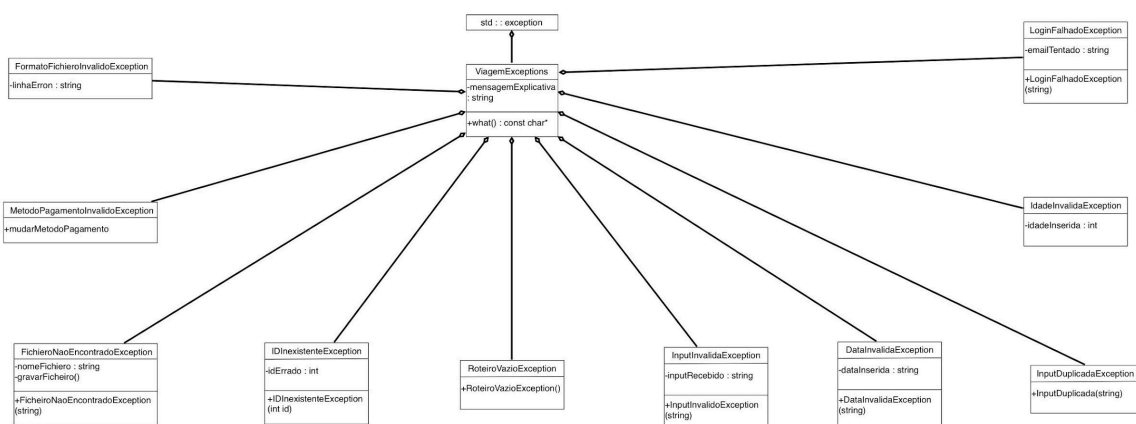
- DataHora/DataViagem/Calendário: Um módulo dedicado ao tratamento temporal, responsável pela validação de datas e cálculos de duração.
- Financeiro: Uma classe de suporte estático que processa os cálculos de base, aplicação de taxas (IVA) e montantes finais, operando sobre as escolhas contidas no roteiro.
- InvalidDataException: Uma classe de exceção personalizada que atravessa todo o modelo, garantindo que qualquer erro de inserção de dados seja capturado e tratado de forma segura.

- Método `what()`: Sobrescreve a função base para exibir a mensagem específica no terminal ou interface.

4.3 Especializações (Derivadas)

A partir da `InvalidDataException`, o diagrama apresenta especializações para cenários específicos:

- `InvalidDateException`: Disparada especificamente pelos objetos da classe `Data` quando ocorrem erros cronológicos (ex: dia 31 em fevereiro ou anos negativos).
- `InvalidTypeException`: Utilizada quando o formato do dado não corresponde ao esperado (ex: inserir letras num campo que espera o preço ou o número de viajantes)



5. Diagrama de Views

O Diagrama de Classes de Views descreve a arquitetura da interface do SGPV, detalhando como o sistema organiza a apresentação de dados e a captura de inputs. Diferente das classes lógicas, as classes de "View" têm a responsabilidade exclusiva de formatar a consola, exibir menus e validar a interação imediata do utilizador antes de comunicar com o núcleo do sistema (Model).

A estrutura está organizada em módulos de responsabilidade, visíveis através das associações no diagrama:

5.1 Gestão de Navegação e Catálogo

- `MainMenuView`: A classe mestre que gere o fluxo principal e a paginação do sistema.
- `CatalogoView` e `FiltroPesquisaView`: Especializadas na renderização de dados provenientes do `RepositorioCatalogo`. Estas classes garantem que a listagem de países e cidades seja apresentada de forma organizada e legível.

5.2 Fluxo de Acesso e Identidade

- **LoginView** e **LogoutView**: Responsáveis pelos formulários de autenticação e registo. Elas interagem diretamente com a classe lógica **Login**, assegurando que o feedback de sucesso ou erro no acesso seja comunicado de forma clara ao utilizador.

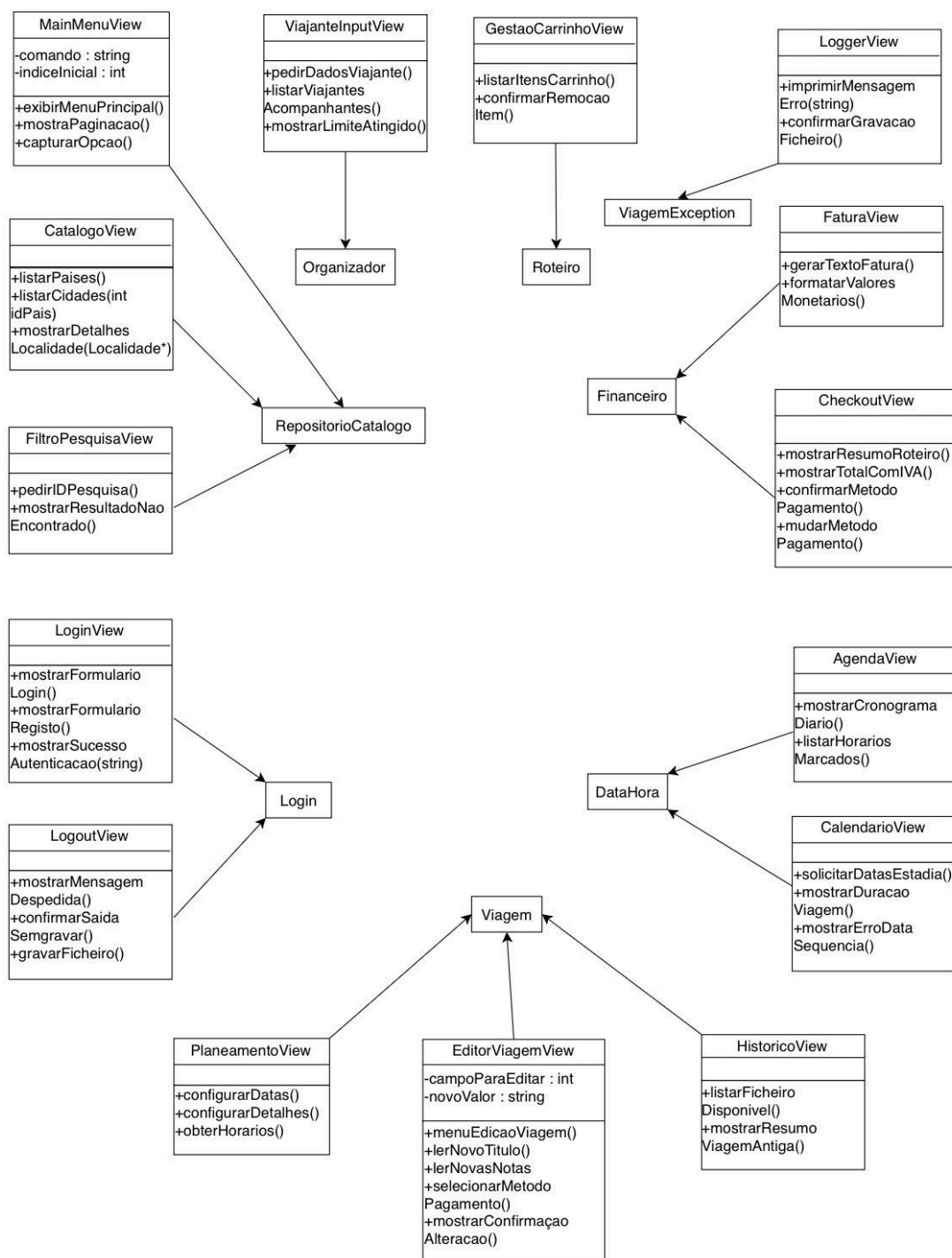
5.3 Planeamento e Edição de Viagem

Este é o núcleo operacional da interface, composto por:

- **PlaneamentoView**: Gere a configuração inicial (datas e detalhes).
- **EditorViagemView**: Permite a modificação dinâmica de atributos já existentes, como o título ou as notas.
- **AgendaView** e **CalendarioView**: Focam-se na representação cronológica e temporal, comunicando com o módulo **DataHora**.

5.4 Finalização e Apoio Financeiro

- **CheckoutView** e **FaturaView**: Transformam os cálculos complexos da classe **Financeiro** em documentos compreensíveis, apresentando resumos de custos, taxas de IVA e confirmações de pagamento.
- **LoggerView**: Uma classe transversal utilizada para imprimir mensagens de sistema, erros críticos e confirmações de gravação em ficheiro, servindo como canal de comunicação de estado (log).



6. Diagrama de controladores

O Diagrama de Classes de Controladores representa o "cérebro" do SGPV. Esta camada é responsável por mediar a comunicação entre as Interfaces (Views) e a Lógica de Negócio

(Model). Os controladores interpretam os comandos do utilizador, decidem quais os dados a carregar e determinam qual a próxima vista a ser apresentada, garantindo um fluxo de navegação lógico e coeso.

A arquitetura de controlo está estruturada de forma hierárquica e modular:

1. Orquestração Central

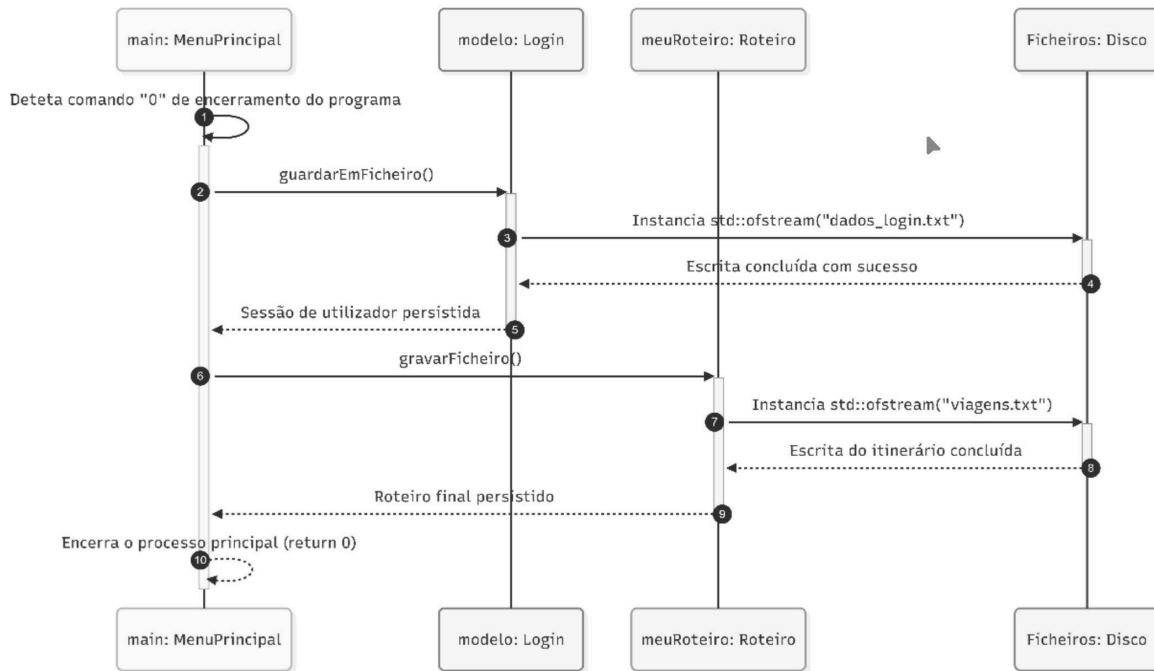
- **MainMenuController**: Atua como o ponto central de decisão. É ele que recebe as opções do menu principal e delega o controlo para os controladores especializados (Login, Catálogo ou Planeamento).
- **LoggerController**: Um componente crítico e transversal. Como se observa pelas múltiplas setas tracejadas, quase todos os controladores comunicam com o **LoggerController** para registar eventos, gerir erros e fornecer feedback centralizado ao utilizador.

2. Gestão de Fluxos Operacionais

- **LoginController** e **LogoutController**: Fazem a ponte entre a autenticação do utilizador e a persistência de sessões.
- **PlaneamentoController**: Coordena a criação de novos roteiros, interagindo com o **ViajanteController**, **AgendaController** e **CalendarioController** para garantir que todos os detalhes temporais e logísticos são capturados corretamente.
- **HistoricoController** e **EditorViagemController**: Gerem o ciclo de vida das viagens já criadas, permitindo a recuperação de ficheiros antigos e a modificação de planos ativos.

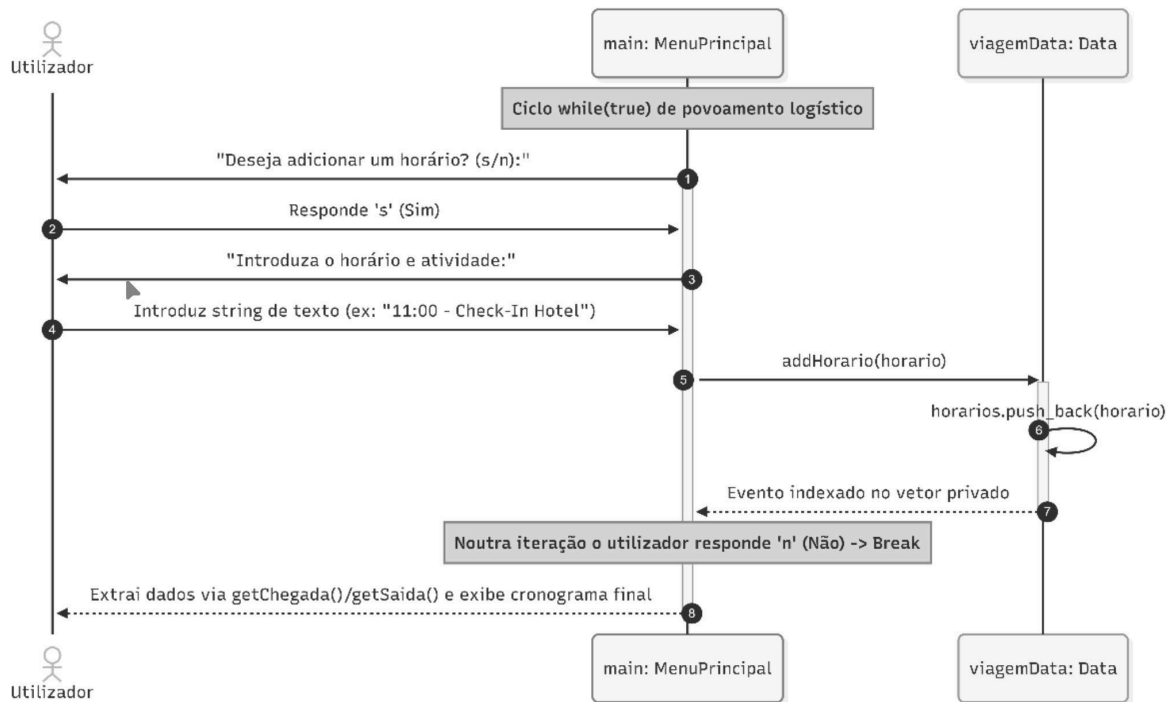
3. Processamento de Catálogo e Venda

- **CatalogoController** e **FiltroViagemController**: Gerem a lógica de exploração de dados, filtrando a informação do repositório antes de a enviar para a vista.
- **GestaoCarrinhoController**, **CheckoutController** e **FaturaController**: Formam a equipa financeira. O **GestaoCarrinhoController** organiza as escolhas, enquanto os restantes processam o fecho do plano e a emissão dos detalhes de pagamento.



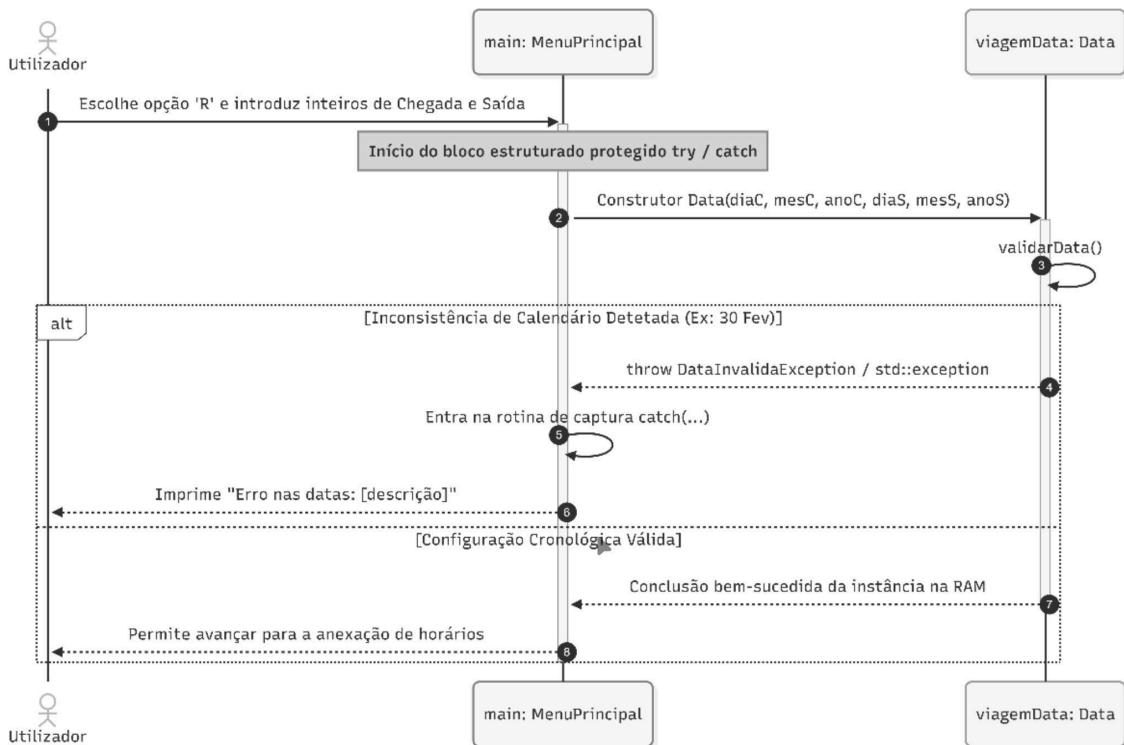
- UC02: Indexar Atividades e Horários Cronológicos

Este caso de uso detalha o processo de organização temporal de uma viagem, onde o utilizador define a agenda específica para cada destino seleccionado. O objetivo central é demonstrar como o sistema gere a inserção de dados descritivos e horários, garantindo que estes fiquem vinculados à instância correta da viagem. Como se pode observar na Figura X(IMG_0989), a sequência foca-se na receção de *inputs* textuais e na sua conversão em elementos estruturados dentro da lista dinâmica de eventos da viagem.



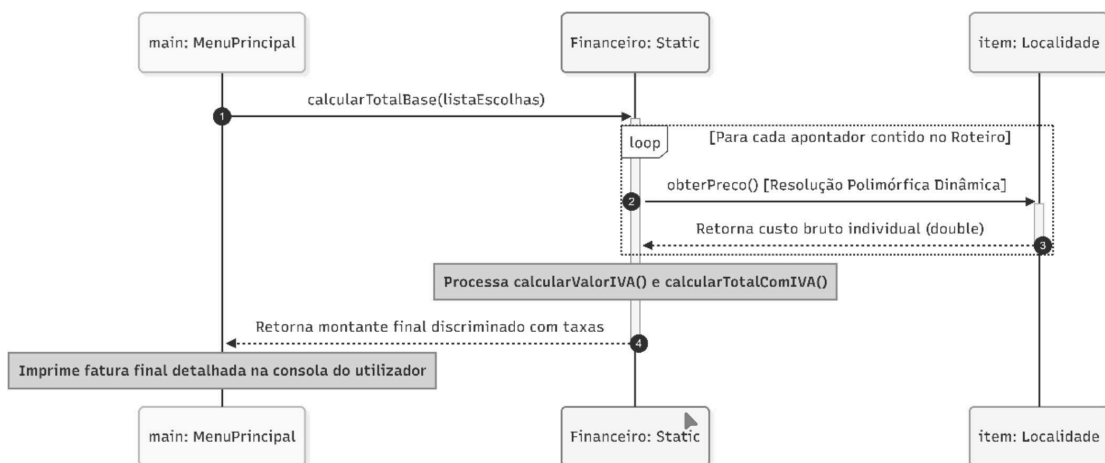
- UC03: Instanciar Período Temporal com Captura de Exceções Logísticas

Este diagrama de sequência foca-se na robustez do sistema durante a definição do intervalo de tempo de uma viagem. O objetivo principal é demonstrar o funcionamento do mecanismo de Captura de Exceções Logísticas, garantindo que o software reaja de forma inteligente a erros de input antes que estes comprometam a lógica de negócio. Como se pode observar na Figura X (IMG_0988), o fluxo destaca o papel do controlador como um validador ativo, impedindo a progressão do programa caso os dados temporais sejam incoerentes.



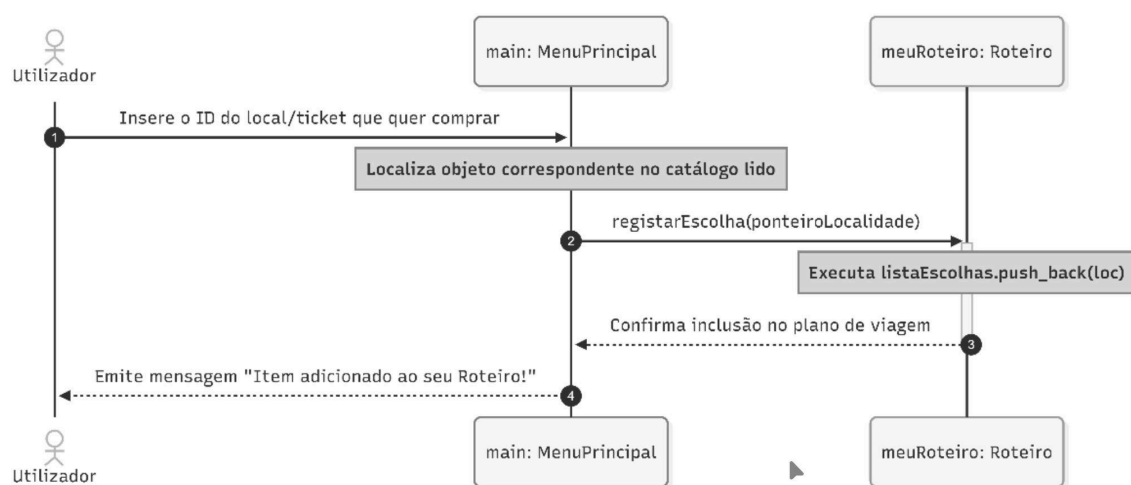
- UC04: Executar Processamento Financeiro e Faturação (Checkout)

Este diagrama de sequência descreve o módulo financeiro do SGPV, representando o momento em que o plano de viagem é convertido num orçamento detalhado. O objetivo é demonstrar a integridade do cálculo de custos, garantindo que o sistema aplica as taxas fiscais corretas de forma automatizada. Como se pode observar na Figura X (IMG_0987), a sequência ilustra a transição da escolha subjetiva de destinos para a objetividade financeira, culminando na geração de uma fatura para o utilizador.



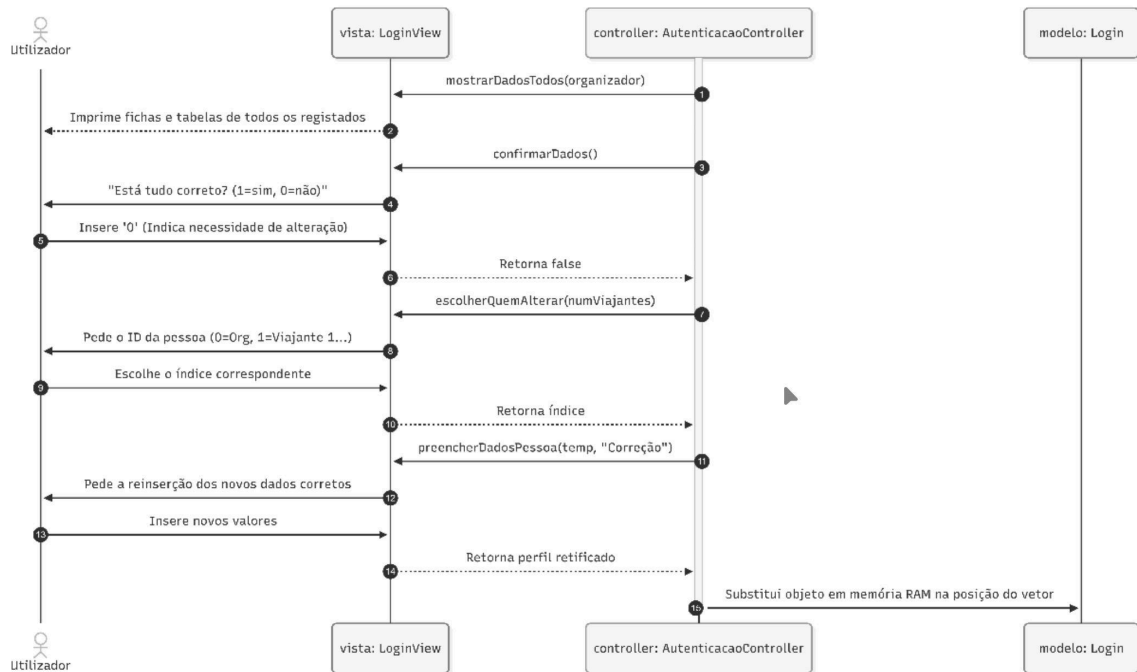
- UC05: Adicionar Elemento ao Roteiro Polimórfico (Carrinho)

Este diagrama de sequência detalha a funcionalidade central de personalização do itinerário, permitindo que o utilizador construa a sua viagem de forma modular. O conceito de "Polimorfismo" é aqui fundamental, pois permite que o sistema trate diferentes tipos de destinos (Cidades, Monumentos ou Atividades) de forma uniforme. Como se pode observar na Figura X (edited-image.png), o fluxo demonstra como o controlador gere a inserção de objetos dinâmicos numa lista de escolhas, mantendo a integridade da estrutura do roteiro.



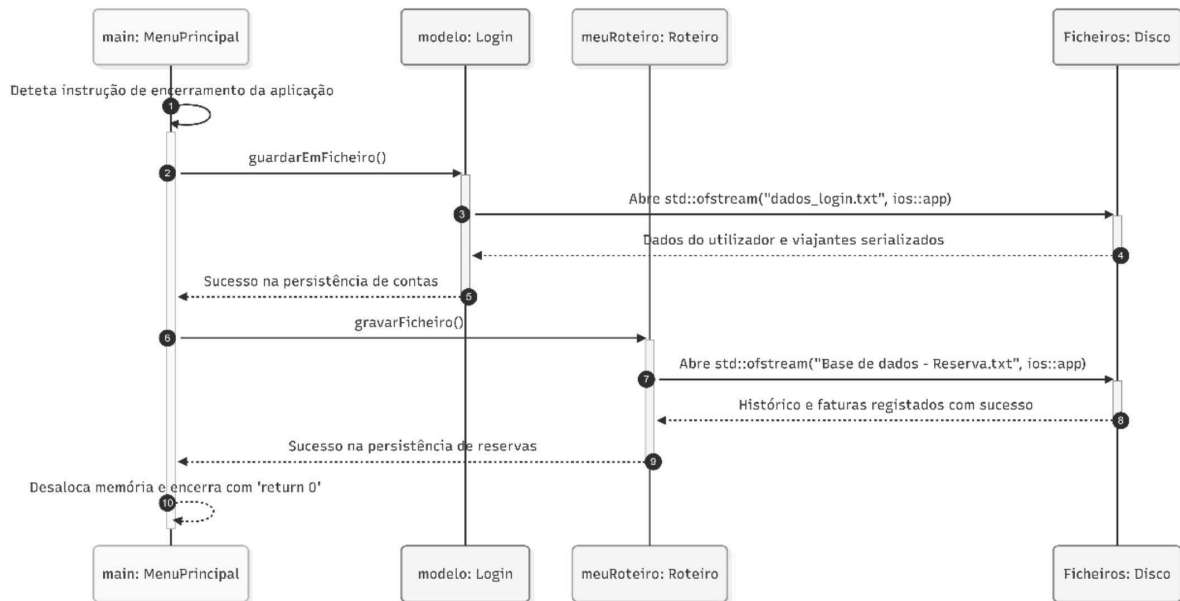
- UC06: Revisão Geral e Edição Polimórfica de Perfis

Este caso de uso descreve a funcionalidade de gestão e auditoria de perfis dentro do sistema. O objetivo é permitir que o Organizador reveja e modifique os dados dos viajantes associados ao plano, utilizando o conceito de Edição Polimórfica. Como se pode observar na Figura X (IMG_0985), a sequência foca-se na capacidade do sistema de aceder a diferentes tipos de perfis através de uma interface comum, garantindo que as atualizações de dados (como nomes, idades ou preferências) sejam refletidas instantaneamente na memória da aplicação.



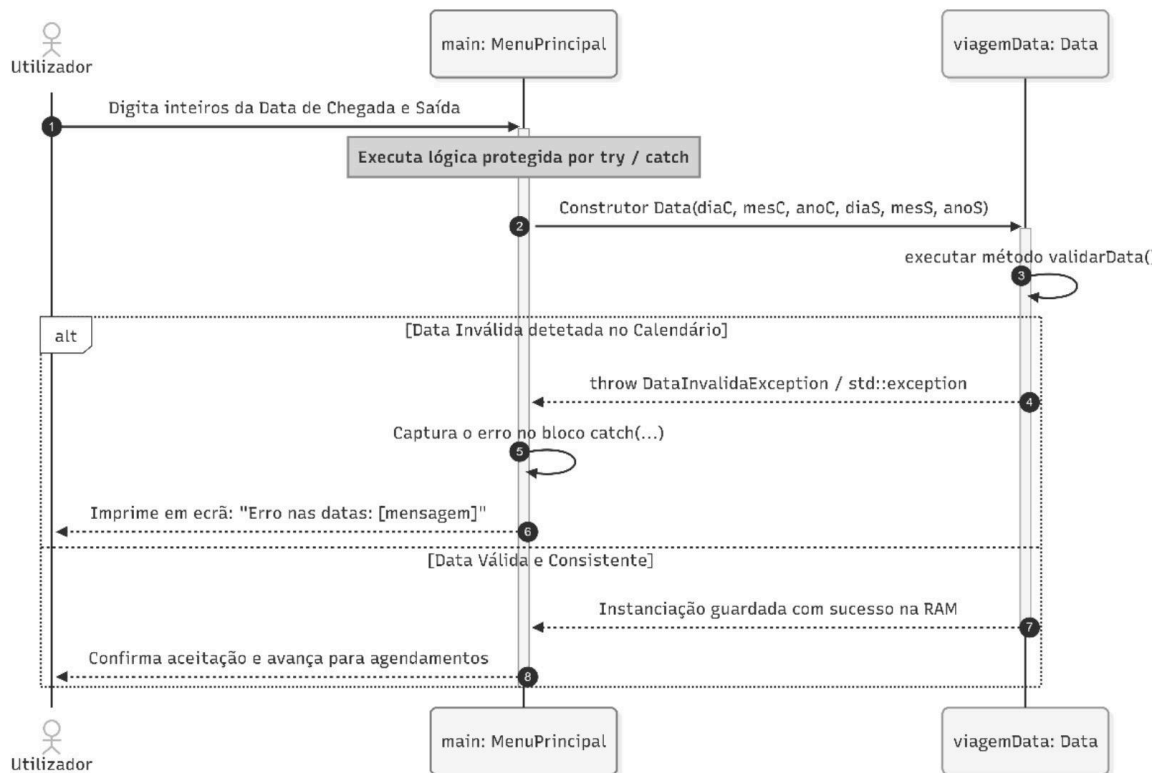
- UC07: Povoar e Associar Lista Dinâmica de Viajantes

Este caso de uso descreve o processo de definição do grupo de viagem, permitindo que o Organizador registre e vincule múltiplos participantes ao plano de viagem. O foco técnico aqui é a gestão de coleções dinâmicas, onde o sistema deve ser capaz de crescer conforme o número de viajantes introduzidos. Como se pode observar na Figura X (IMG_0984), a sequência ilustra a transição de dados individuais para uma estrutura de lista agregada, garantindo que cada perfil de viajante esteja corretamente associado ao nó principal do Organizador.



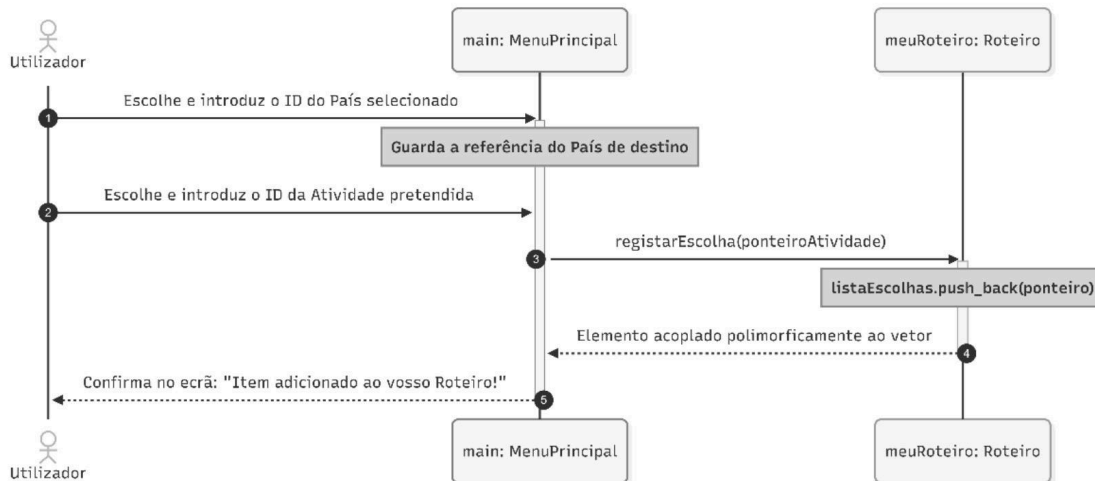
- UC08: Autenticação de Credenciais (Login de Conta)

Este diagrama de sequência ilustra a "porta de entrada" do sistema, detalhando o protocolo de segurança para a autenticação de utilizadores. O objetivo é demonstrar como o sistema valida a identidade do utilizador antes de conceder acesso aos planos de viagem privados. Como se pode observar na Figura X (IMG_0983), a sequência foca-se no processo de verificação cruzada entre os dados inseridos na interface e os registos armazenados na base de dados persistente do sistema.



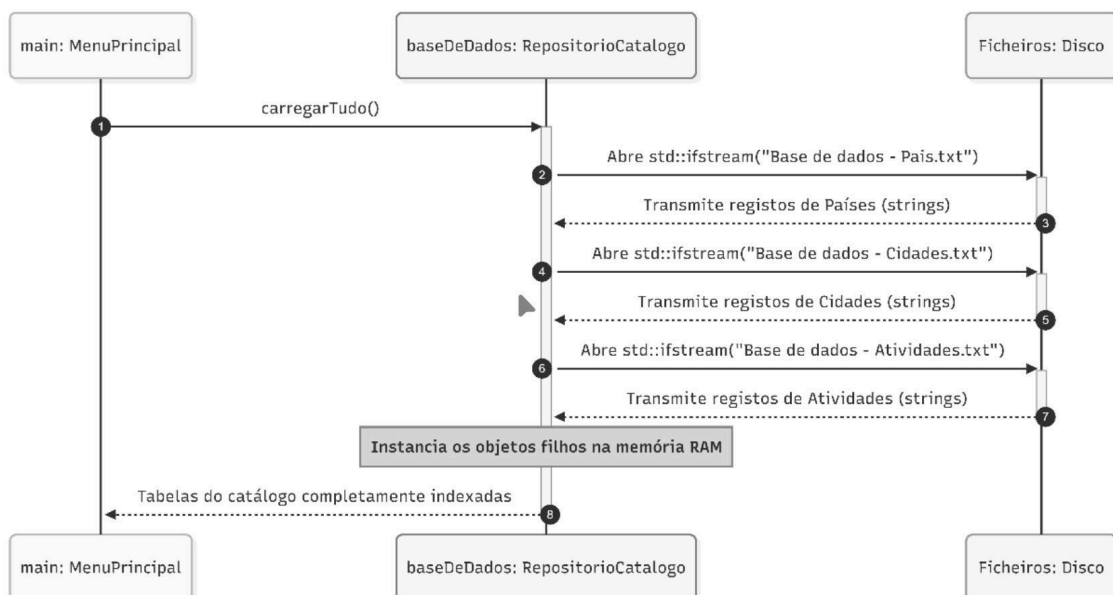
- UC09: Registrar Organizador com Controlo de Menoridade Legal

Este diagrama de sequência descreve o fluxo de criação de conta para o perfil principal do sistema: o Organizador. A particularidade técnica deste caso de uso reside no mecanismo de Validação de Maioridade, uma regra de negócio crítica que impede que indivíduos sem capacidade jurídica plena assumam a responsabilidade por planos de viagem. Como se pode observar na Figura X (IMG_0982), o sistema atua como um inspetor de dados, validando a elegibilidade do utilizador antes de permitir a persistência do perfil na base de dados.



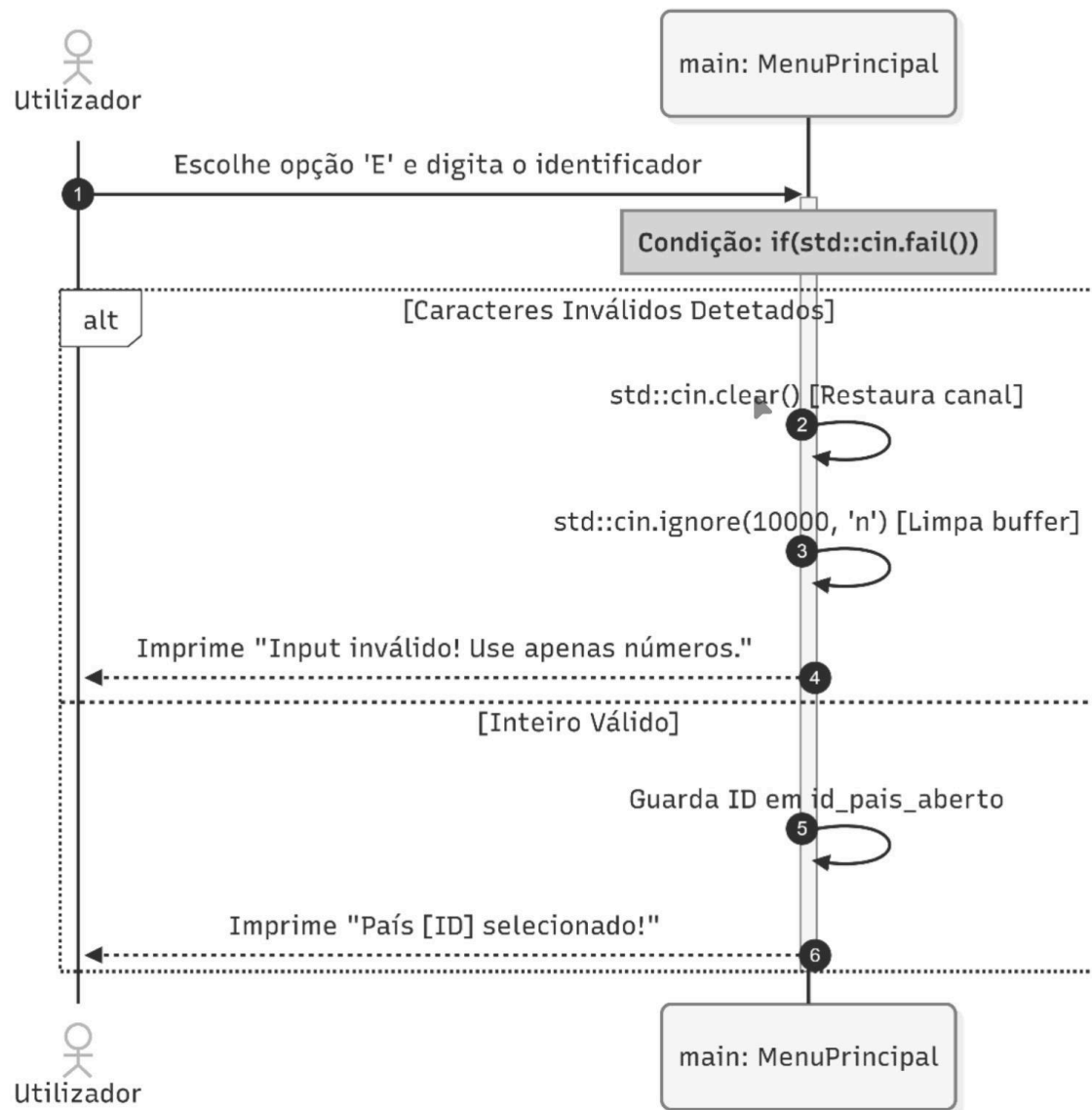
- UC10: Carregar Base de Dados do Catálogo (Inicialização do Sistema)

Este diagrama de sequência ilustra o procedimento de *bootstrapping* ou inicialização do SGPV, focando-se na transição de dados do armazenamento estático para o ambiente de execução dinâmico. O objetivo é demonstrar como o sistema popula o catálogo de destinos (Países, Cidades e Atividades) logo após o arranque, garantindo que o utilizador tenha informação disponível para consulta imediata. Como se pode observar na Figura X (IMG_0981), o fluxo destaca a interação entre o motor de arranque do sistema e a camada de persistência de ficheiros.



- UC11: Selecionar País com Validação de Tipo de Dados (Input Shield)

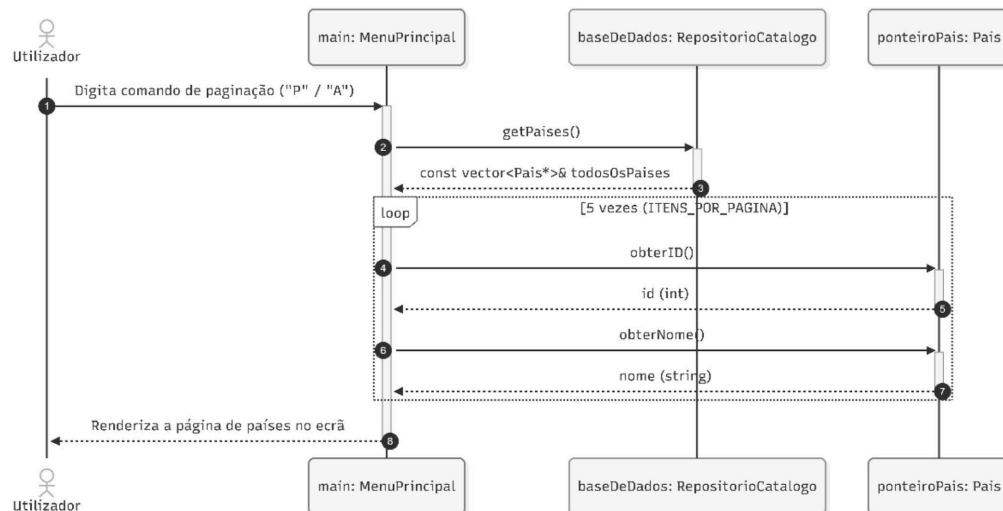
Este diagrama de sequência detalha o processo de interação segura entre o utilizador e o catálogo do sistema. O foco principal é o mecanismo de Input Shielding, que atua como uma barreira de proteção na camada da View. O objetivo é garantir que apenas comandos válidos cheguem à lógica de controlo, prevenindo falhas comuns de conversão de dados. Como se pode observar na Figura X (IMG_0980), a sequência ilustra como o sistema valida a natureza do input antes de tentar processar a seleção de um país.



- UC12: Consultar Catálogo Paginado de Países

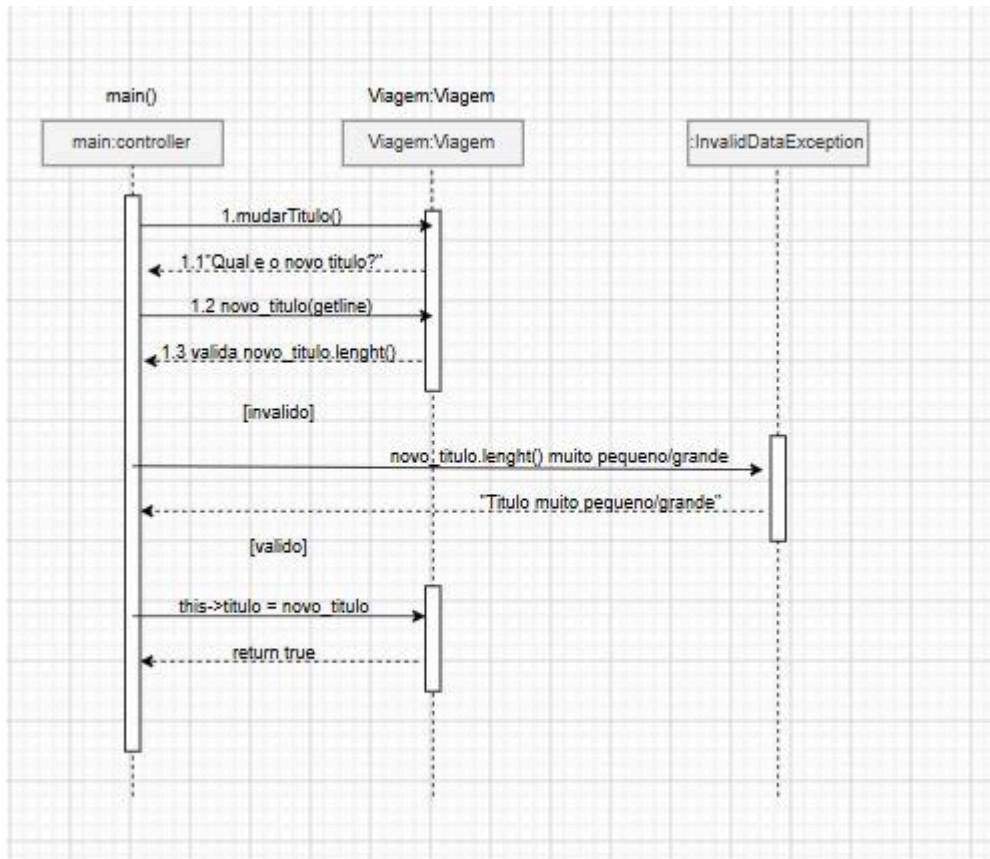
Este diagrama de sequência ilustra a funcionalidade de exploração do catálogo, focando-se na eficiência da apresentação de dados através do mecanismo de Paginação Dinâmica. O objetivo é demonstrar como o sistema recupera coleções de objetos da memória e os organiza em "páginas" numeradas, facilitando a navegação

do utilizador. Como se pode observar na Figura X (IMG_0979), a sequência destaca a separação entre o armazenamento total de dados e a renderização parcial no ecrã, garantindo uma interface limpa e performante.



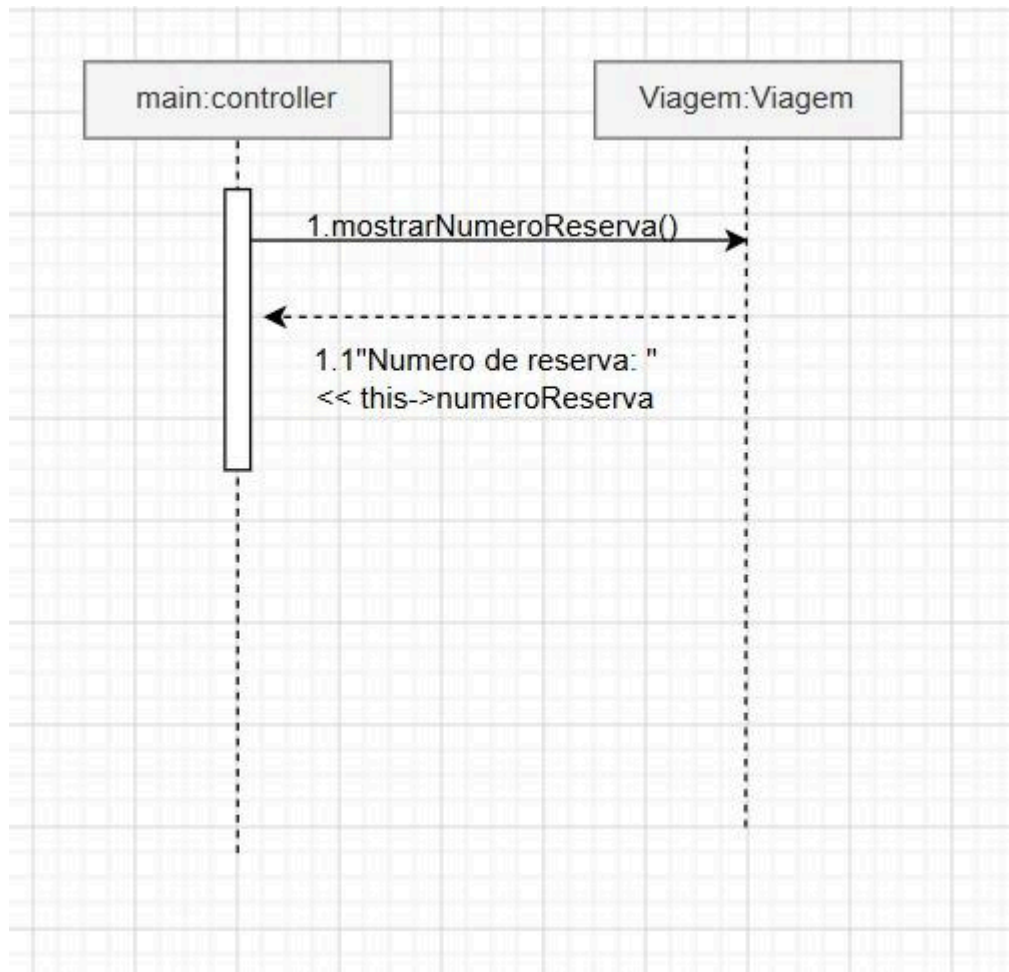
- UC13: Alterar Designação do Plano com Validação de Extensão (Título)

Este diagrama de sequência ilustra o processo de edição de metadados de uma viagem, especificamente a alteração do seu título. O foco técnico reside na Integridade de Atributos, onde o sistema valida o comprimento da *string* inserida antes de permitir a alteração na memória RAM. Como se pode observar na Figura X (1779221859720.jpeg), o fluxo utiliza um mecanismo de captura de exceções para garantir que o título respeita os limites de usabilidade definidos no código, evitando títulos vazios ou excessivamente longos que possam comprometer a interface.



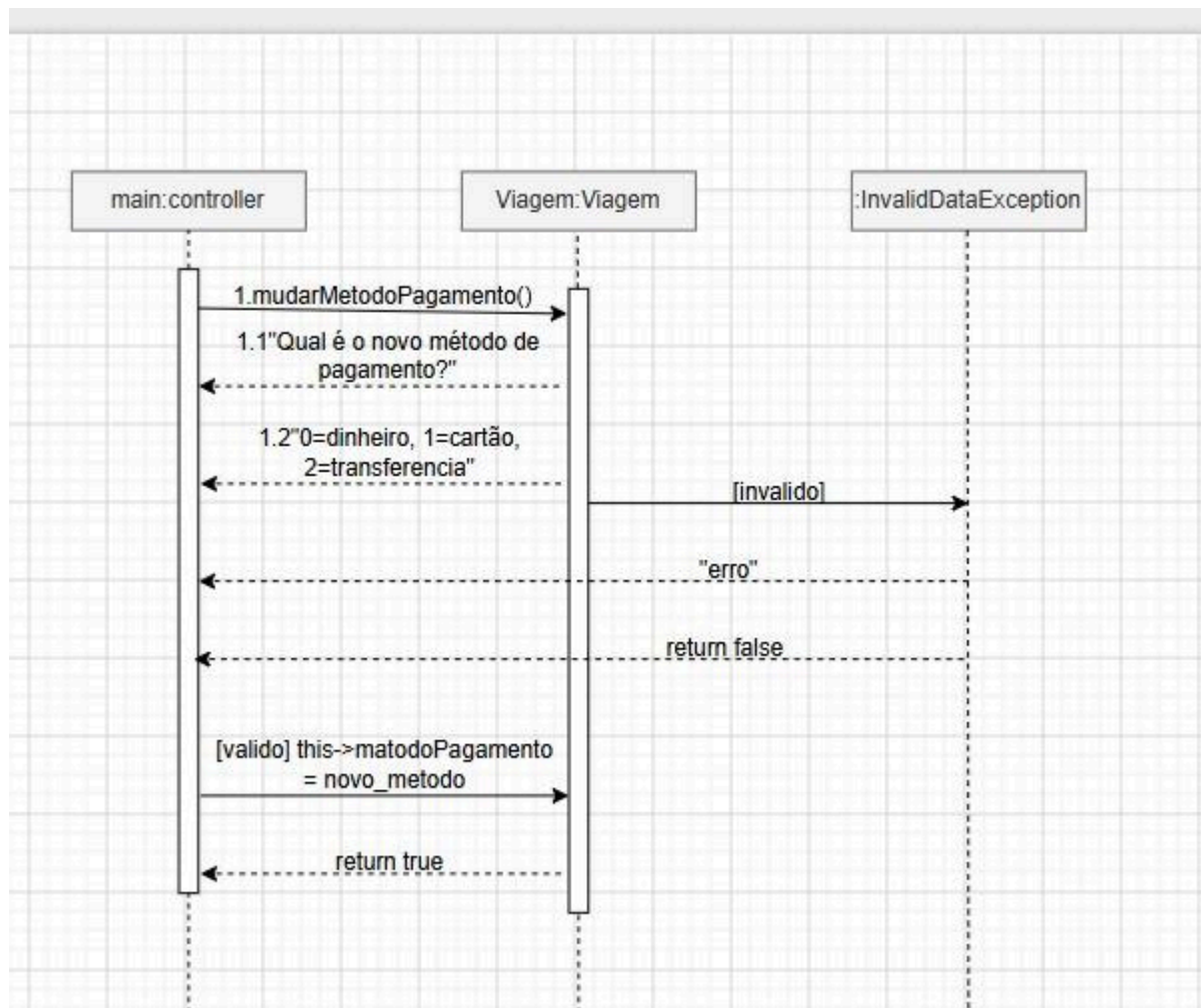
- UC14: Consulta de Identificador Único de Reserva

Este diagrama de sequência descreve uma funcionalidade de consulta direta e essencial para o utilizador: a recuperação do número de reserva associado ao seu plano de viagem. O objetivo deste caso de uso é demonstrar a eficiência do sistema no acesso a atributos específicos de uma instância, garantindo que o utilizador tenha sempre acesso rápido à sua referência de confirmação. Como se pode observar na Figura X (1779221859565.jpeg), o fluxo é desenhado de forma linear para minimizar a latência entre o pedido do utilizador e a resposta do sistema.



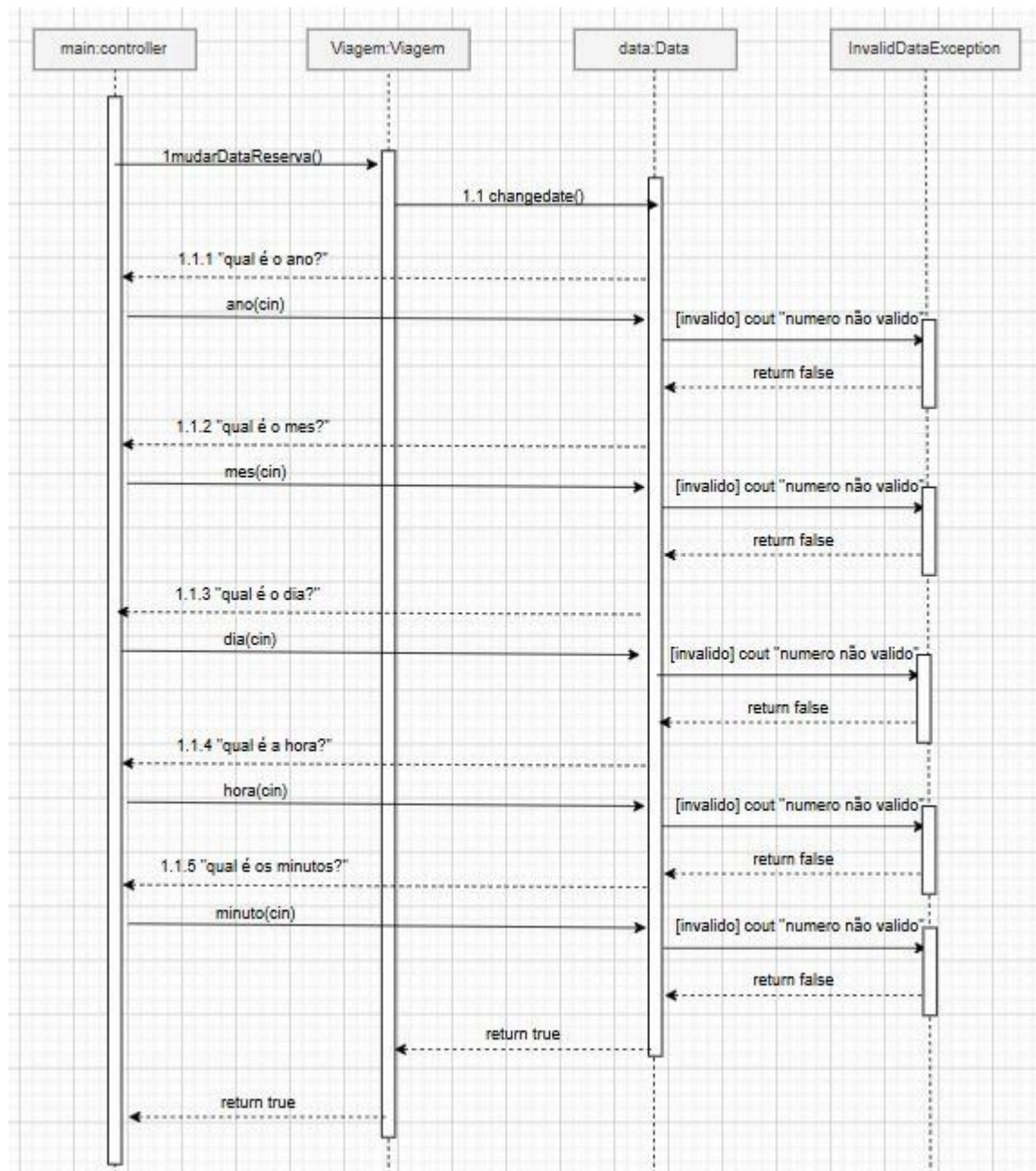
- UC15: Alteração do Método de Pagamento com Tratamento de Exceções

Este diagrama de sequência detalha a funcionalidade de gestão financeira que permite ao utilizador definir ou alterar a forma como tenciona liquidar o custo da viagem. O objetivo central é demonstrar a Robustez do Sistema de Input, garantindo que apenas opções de pagamento suportadas pela lógica de negócio sejam aceites. Como se pode observar na Figura X (1779221859535.jpeg), o fluxo destaca a proteção contra seleções inválidas através do lançamento de exceções, assegurando que o estado financeiro da viagem permaneça sempre consistente.



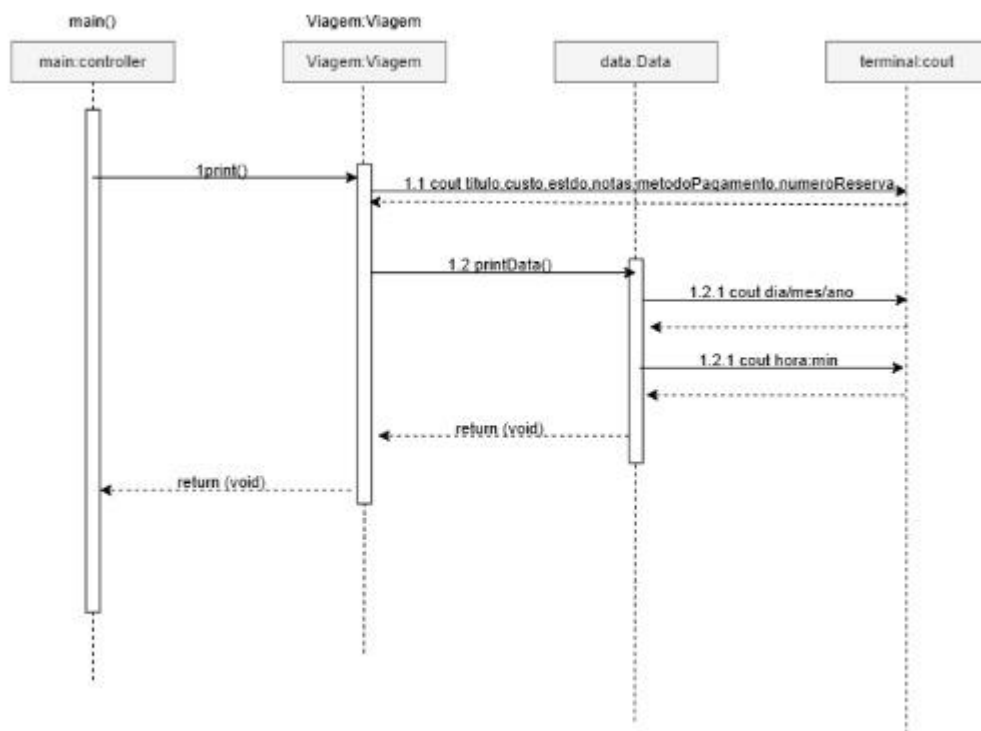
- UC16: Alteração da Data e Hora de Reserva com Validação

Este diagrama de sequência descreve o protocolo de modificação temporal de uma reserva existente. Diferente de uma simples inserção de texto, este caso de uso exige uma Validação de Atributos Múltiplos, onde cada componente da data (ano, mês, dia) e da hora (horas, minutos) é verificado individualmente. Como se pode observar na Figura X(1779221859660.jpeg), o sistema utiliza uma estrutura de diálogo interativo que impede a progressão do fluxo caso qualquer um dos valores inseridos seja logicamente impossível, garantindo a integridade cronológica do SGPV.



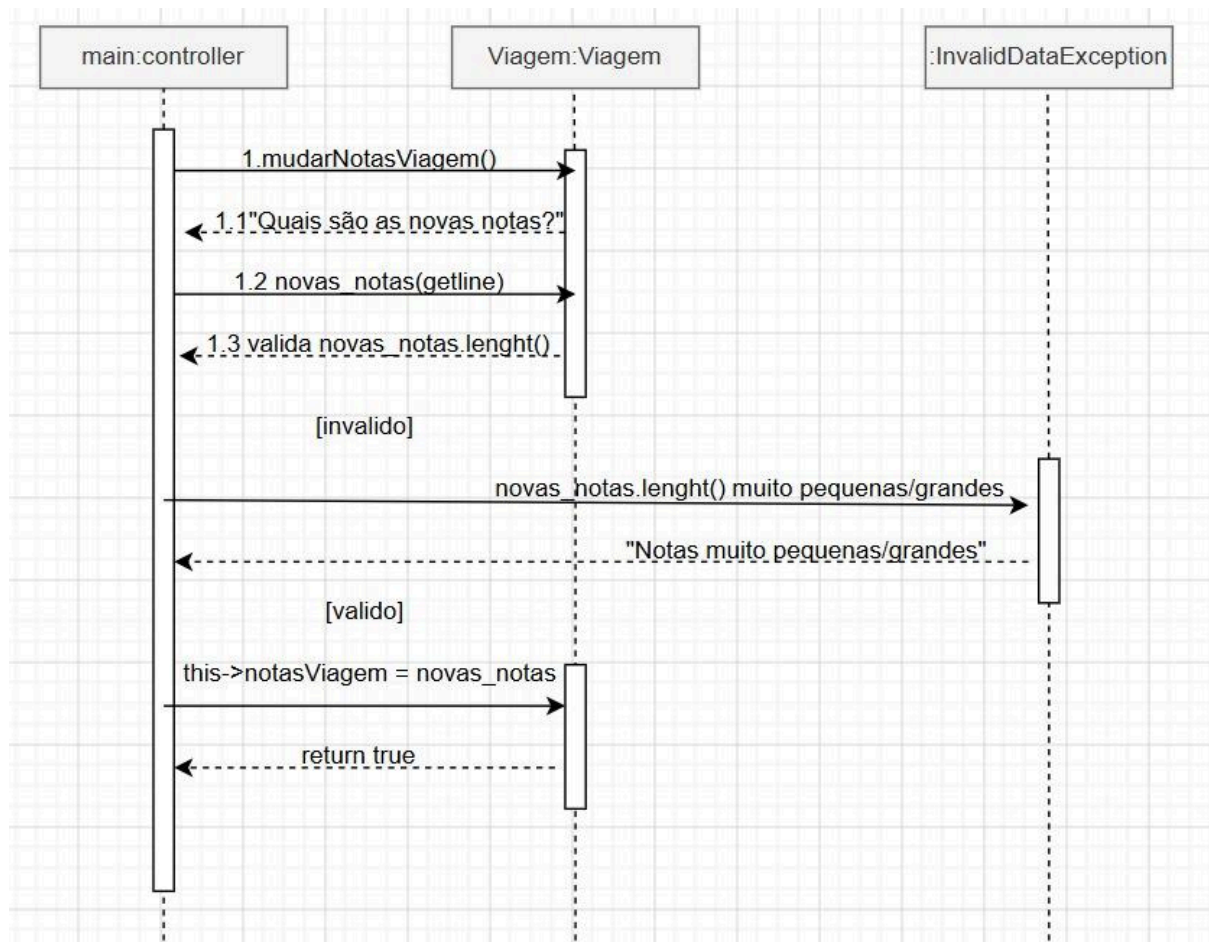
- UC17: Gerar Relatório Consolidado de Viagem

Este diagrama de sequência descreve o processo de saída de dados (*Output*) final do sistema. O objetivo é demonstrar a capacidade de Agregação de Dados, onde diferentes objetos e atributos são chamados em cascata para formar um resumo legível e completo da viagem. Como se pode observar na Figura X (1779221859690.jpeg), o fluxo ilustra a cooperação entre a classe principal e as suas classes auxiliares (como a Data) para garantir que nenhuma informação relevante seja omitida na visualização final.



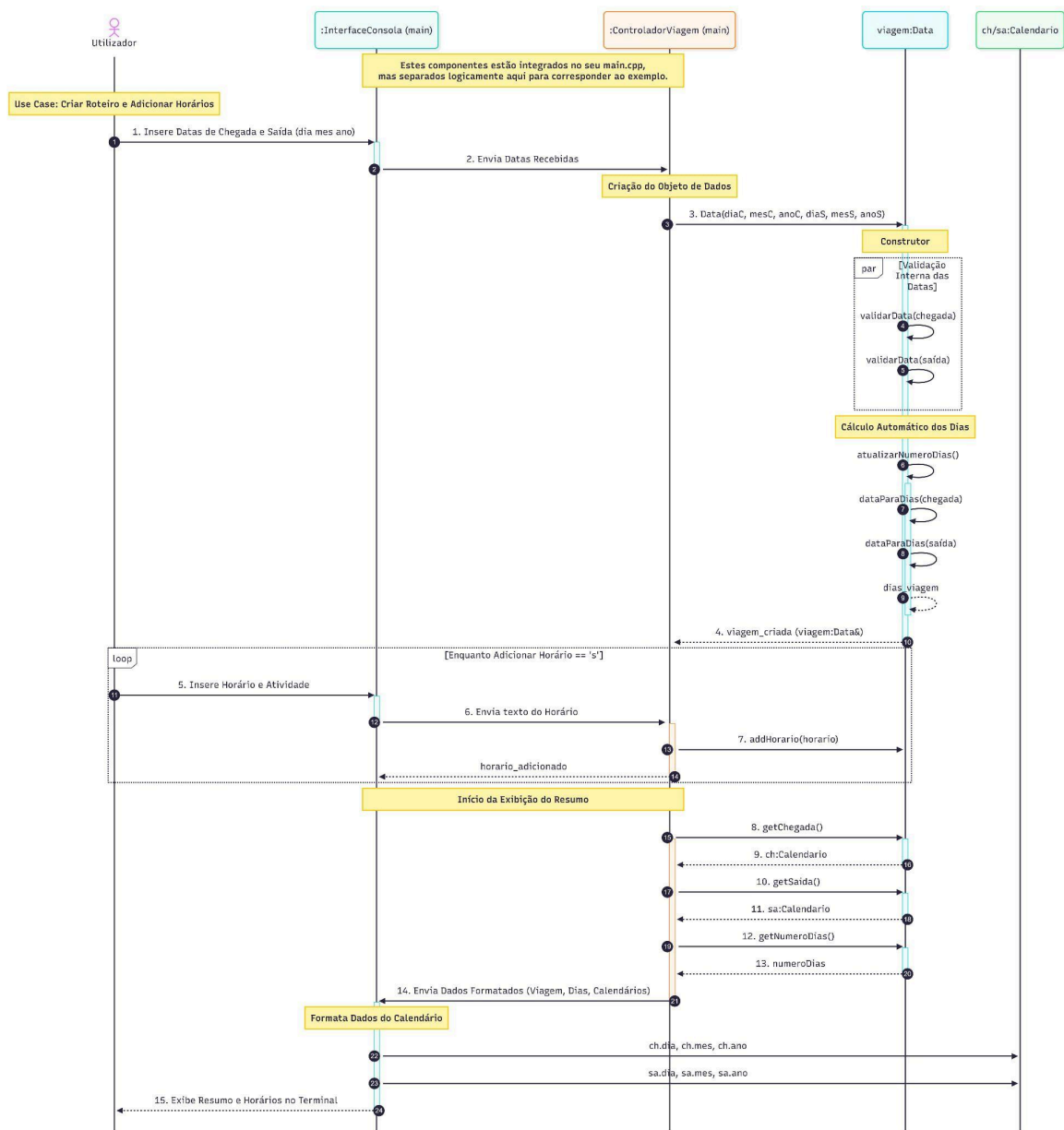
- UC18: Alteração de Notas da Viagem com Validação de Extensão

Este diagrama de sequência descreve a funcionalidade que permite ao utilizador anexar ou modificar observações personalizadas ao seu plano de viagem (ex: "Levar casaco quente" ou "Confirmar transfer"). Tal como na edição do título, este processo não é apenas uma inserção direta de texto; ele inclui uma camada de Validação de Integridade para garantir que o campo de notas não seja utilizado de forma indevida (ex: textos vazios ou demasiado extensos que possam quebrar a formatação do relatório final). Como se pode observar na Figura X (1779221859596.jpeg), o sistema utiliza um bloco de proteção para validar o input antes da gravação em memória.



- UC19: Criar Roteiro e Adicionar Horários (Processamento Cronológico)

Este diagrama de sequência detalha o núcleo logístico do sistema: o momento em que o utilizador define a moldura temporal da sua viagem. O foco aqui é o Cálculo Automático de Duração e a Validação Interna de Calendário. Diferente de outros fluxos, este demonstra uma interação complexa entre quatro componentes distintos, garantindo que as datas de chegada e saída são logicamente possíveis e que o roteiro é construído sobre uma base temporal sólida. Como se pode observar na Figura X (1779221859749.jpeg), o sistema não se limita a guardar números, mas processa-os para gerar informação útil, como a contagem total de dias.



8. GRASP e SOLID

8.1 SOLID

S - Single Responsibility Principle (Princípio da Responsabilidade Única)

Uma classe deve ter apenas um motivo para mudar, ou seja, deve ter apenas uma responsabilidade.

- Exemplo

```
class Financeiro {
```

public:

```
static double calcularTotalBase(const std::vector<Localidade*>& lista);
```

```
static double calcularIVA(double totalBase, double taxa);
```

```
static double calcularTotalFinal(double totalBase, double iva);
```

```
};
```

- Esta classe não sabe como ler ficheiros, não sabe imprimir menus no ecrã e não guarda listas de viagens. A sua única responsabilidade é processar números (calcular totais e IVA).

O - Open/Closed Principle (Princípio do Aberto/Fechado)

As classes devem estar abertas para extensão, mas fechadas para modificação.

- Exemplo:

```
// No Localidade.h
```

```
virtual double obterPreco() const;
```

```
// No Roteiro.cpp
```

```
void Roteiro::registarEscolha(Localidade* localidade) {
```

```
    listaEscolhidos.push_back(localidade);
```

```
}
```

- Criamos uma classe base Localidade de onde derivam Pais , Cidade e Atividade. Se amanhã o professor pedir para adicionar um tipo "Restaurante" ou "Voo", tu não precisas de modificar a classe Roteiro nem a classe Financeiro. Basta criares a nova classe Voo : public Localidade e implementares o obterPreco(). O sistema está aberto a novas extensões sem alterar o código existente.

L - Liskov Substitution Principle (Princípio da Substituição de Liskov)

As classes filhas devem poder substituir as classes base sem quebrar o funcionamento do programa.

- Exemplo:

```
// Roteiro.h / .cpp
```

```
void Roteiro::registarEscolha(Localidade* localidade) {
```



```
listaEscolhidos.push_back(localidade);  
}
```

- O nosso roteiro aceita um ponteiro genérico para Localidade. Em tempo de execução, podes passar-lhe um objeto Pais, uma Cidade ou uma Atividade. O programa continua a funcionar perfeitamente quando chama `ponteiroLocalidade->obterPreco()`, provando que qualquer classe derivada se comporta corretamente como o seu tipo base.

I - Interface Segregation Principle (Princípio da Segregação de Interfaces)

Uma classe não deve ser forçada a implementar interfaces/métodos que não vai utilizar.

- Exemplo:

```
virtual void mostrarDados() = 0;
```

```
virtual void carregarDeLinha(std::string linha) = 0;
```

- Temos um `virtual void mostrarDados() = 0;` e `virtual void carregarDeLinha(std::string linha) = 0;`, obrigaste as classes filhas a implementar apenas os comportamentos estritamente necessários para que existam no catálogo. Não criaste métodos "gordos" que as subclasses teriam de deixar em branco.

D - Dependency Inversion Principle (Princípio da Inversão de Dependência)

Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

- Exemplo:

```
double Financeiro::calcularTotalBase(const std::vector<Localidade*>& lista) {  
  
    double total = 0;  
  
    for (auto const& item : lista) {  
  
        total += item->obterPreco(); // Inversão de Dependência e Polimorfismo  
  
    }  
  
    return total;  
  
}
```

- A classe de alto nível Financeiro não depende das classes concretas Atividade ou Cidade. Ela depende apenas da abstração Localidade*. O teu próprio

comentário no código diz isto na perfeição: *"Mesmo que a lista seja de 'Localidade', o C++ sabe se deve ir buscar o preço da Cidade ou da Atividade".*

8.2 GRASP

- Information Expert (Especialista na Informação)

Atribuir a responsabilidade à classe que possui a informação necessária para a cumprir.

- Exemplo:

```
bool DataHora::validarData(int d, int m, int a) {  
    if (a < 2024) return false;  
    if (m < 1 || m > 12) return false;  
    if (d < 1 || d > diasNoMes(m, a)) return false;  
    return true;  
}
```

- Porquê: Quem é que valida se fevereiro tem 28 ou 29 dias? É o método `validarData()` dentro da `DataHora`. Porquê? Porque é esta classe que detém as variáveis `dia`, `mes` e `ano`. Ela é a "especialista" na sua própria informação.

- Creator (Criador)

Quem deve ser responsável por criar uma nova instância de uma classe?

- Exemplo:

```
if (tipo == "PAIS") {  
    Pais* p = new Pais();  
    p->carregarDeLinha(linha);  
    listaPaises.push_back(p);  
}
```

- No método `lerFicheiro`, o repositório faz `Pais* novo = new Pais();` e `Cidade* novo = new Cidade();`. Como o Repositório é o componente que vai agregar e guardar estas listas na memória RAM (`vector<Pais*>`), é a ele que compete a responsabilidade de as criar.

- Polymorphism (Polimorfismo)

Quando o comportamento varia de acordo com o tipo, atribua a responsabilidade usando operações polimórficas.

- Exemplo:

```
double Financeiro::calcularTotalBase(const std::vector<Localidade*>& lista) {  
  
    double total = 0;  
  
    for (auto const& item : lista) {  
  
        total += item->obterPreco();  
  
    }  
  
    return total;  
  
}
```

- A classe base diz que o preço retorna 0.0 , mas a classe Cidade faz um *override* para retornar preco_cidade e a classe Atividade retorna preco_ticket. O cálculo financeiro aciona este polimorfismo dinamicamente.

- Pure Fabrication (Fabricação Pura)

Criar uma classe artificial (que não representa um conceito direto do mundo real) para manter a alta coesão e o baixo acoplamento.

- Exemplo:

```
class Financeiro {  
  
public:  
  
    static double calcularTotalBase(const std::vector<Localidade*>& lista);  
  
    static double calcularIVA(double totalBase, double taxa);  
  
    static double calcularTotalFinal(double totalBase, double iva);  
  
};
```

- No mundo real de uma agência de viagens, existem "Roteiros", "Pessoas" e "Cidades". Um "Financeiro" não é um objeto palpável da viagem. Tu "fabricaste" esta classe para evitar misturar contas matemáticas dentro da classe Roteiro. Isto salvou a classe Roteiro de ficar com demasiadas responsabilidades (salvando o princípio de Baixo Acoplamento).

- Controller (Controlador)

Atribuir a responsabilidade de lidar com eventos do sistema a uma classe que represente o cenário global.

- Exemplo:

```
while (opcao != "0") {
    std::cout << "1. Ver Catalogo\n2. Planear Viagem...";
    std::cin >> opcao;
    if (opcao == "1") { /* chama funções do catálogo */ }
}
```

- O `main()` atua como um controlador de fachada (Front Controller). É ele que deteta o *input* do utilizador (ex: introduzir o ID do país) e decide para onde encaminhar a ação (para o Catálogo, para as Datas, etc.).

- High Cohesion (Alta Coesão):

- Exemplo:

```
class Login {
private:
    std::string username;
    std::string password;
public:
    bool verificarCredenciais(std::string u, std::string p);
    void registarUtilizador(std::string u, std::string p);
    void salvarFicheiro();
};
```

- Esta classe tem Alta Coesão porque tudo o que está lá dentro serve apenas um propósito: Segurança/Acesso. Ela não tenta calcular preços, não tenta mostrar países. Se precisares de alterar a forma como os utilizadores são guardados, só mexes aqui.

- Low Coupling (Baixo Acoplamento)

- Exemplo:

```
// No Roteiro.h
```

```
#include "Localidade.h"
```

```
#include <vector>
```

```
class Roteiro {
```

```
private:
```

```
    std::vector<Localidade*> listaEscolhidos; // Acoplamento à base, não às filhas
```

```
public:
```

```
    void registrarEscolha(Localidade* localidade);
```

```
};
```

- O Roteiro tem Baixo Acoplamento porque ele só conhece a classe Localidade. Este não conhece as classes Pais, Cidade ou Atividade diretamente. No ficheiro Atividade.cpp, se for mudado o nome de uma variável ou a lógica de um método, o ficheiro Roteiro.cpp não precisaria de ser recompilado nem alterado.

9. Implementação e Boas Práticas de POO

A transição dos modelos arquiteturais teóricos para código C++ funcional foi pautada pela aplicação rigorosa das boas práticas de Programação Orientada a Objetos (POO). A organização estrutural, a segurança de memória e a legibilidade do código foram priorizadas desde o início, de forma a facilitar o trabalho colaborativo da equipa e a garantir a manutenção futura do sistema da Agência de Viagens.

9.1 Regras Estruturais:

Para garantir uma compilação modular eficiente, evitar dependências circulares e manter o projeto organizado no ambiente CLion, o código foi estritamente dividido em dois tipos de ficheiros:

- Diretório de Headers (.h): Ficheiros dedicados exclusivamente à declaração das assinaturas de classes, atributos e protótipos de métodos. Representam a "interface pública" ou o contrato do módulo.
- Diretório de Sources (.cpp): Ficheiros que contêm a implementação lógica e algorítmica real dos métodos previamente declarados.

Exemplo - Separação da classe Localidade:

- Ficheiro heathers:

```
#ifndef LOCALIDADE_H
#define LOCALIDADE_H
#include <string>
```

```
class Localidade {
protected:
    int id;
    std::string nome;

public:
    Localidade();
    int obterID() const;
};
#endif
```

- Ficheiro .cpp:

```
#include "Localidade.h"
```

```
Localidade::Localidade() : id(0), nome("") {
}
```

```
int Localidade::obterID() const {
    return id;
}
```

Como demonstrado acima, a classe Localidade tem a sua "montra" (interface) declarada no ficheiro .h, enquanto a sua "fábrica" (lógica de funcionamento) está encapsulada e escondida no ficheiro .cpp. Esta prática melhora significativamente o tempo de compilação do projeto gerido pelo CMake.

9.2 Encapsulamento de Dados

Um dos pilares deste projeto é a proteção da integridade dos dados (Encapsulamento). Todos os atributos críticos das nossas classes foram declarados sob os modificadores de acesso `private` ou `protected`, impedindo que o estado de um objeto seja adulterado diretamente de forma externa (por exemplo, a partir do `main`). A comunicação com o exterior faz-se exclusivamente através de métodos *getters* (leitura) e *setters* (escrita).

Exemplo - Encapsulamento na classe Pessoa:

- Ficheiro `heathers`:

```
class Pessoa {  
protected:  
    std::string nome;  
    int idade;  
  
public:  
    std::string obterNome() const {  
        return nome;  
    }  
  
    void colocarIdade(int i) {  
        idade = i;  
    }  
};
```

No nosso sistema, a classe Pessoa (e as suas derivadas `Organizador` e `Viajante`) aplica o encapsulamento estrito. Qualquer leitura de dados tem de passar por métodos como `obterNome()`, que utilizam a cláusula `const` para garantir que o estado do objeto se mantém inviolável e *read-only* durante as operações de visualização de dados.

9.3 Convenções de Nomenclatura

De modo a uniformizar a escrita de código entre os vários elementos do grupo e mitigar conflitos durante a integração dos vários módulos, estabeleceram-se regras de notação rigorosas usando o padrão *CamelCase*:

- **UpperCamelCase:** Utilizado para nomear todas as Classes e Exceções Customizadas. A primeira letra de cada palavra é sempre maiúscula.

- Exemplo:

```
class Localidade { ... };
```

```
class Pais : public Localidade { ... };
```

```
class RepositorioCatalogo { ... };
```

```
class InvalidDataException : public std::exception { ... };
```

- **lowerCamelCase:** Utilizado sistematicamente para atributos instanciados, variáveis locais e funções/métodos. A primeira palavra inicia sempre em letra minúscula e as seguintes são capitalizadas.

- Exemplo:

```
std::string notasViagem;
```

```
int idUnico;
```

```
int obterID() const;
```

```
void registrarEscolha(Localidade* localidade);
```

```
void carregarDeLinha(std::string linha);
```

```
double totalBase = 0;
```

```
std::string novaDesignacao;
```

10. Gestão de erros

De maneira a assegurar a estabilidade da aplicação e prevenir falhas de execução resultantes de uma possível introdução incorreta de dados, implementamos um mecanismo de validação contínua, através da função `getSafeInt`. Isto faz com que sempre que o utilizador insira caracteres inválidos, esta função efetue a gestão do buffer de entrada, o que garante que a aplicação apenas prossegue quando os tipos de dados inseridos correspondem ao formato esperado no terminal.

Adicionalmente, o tratamento dos erros foi estruturado com recurso a exceções personalizadas onde é exemplo a `DataInvalidaException`, classe esta que serve para sinalizar erros durante a execução do programa. Adicionou-se ainda uma outra responsável por sinalizar as avaliações que ultrapassam os limites predefinidos pelo domínio da aplicação, que é o caso da `InvalidRatingException`.

```
class DataInvalidaException : public std::exception {
private:
```



```

    std::string mensagem;
public:
    DataInvalidaException(std::string msg) : mensagem(msg) {}
    const char* what() const noexcept override {
        return mensagem.c_str();
    }
};

```

Na camada de interface (ou apresentação), a interação do organizador com o sistema é protegida através dos blocos try-catch. Estes blocos permitem interceptar as exceções disparadas pela camada lógica e ao invés de ocorrer um encerramento abrupto do programa, o erro é devidamente apresentado numa mensagem de aviso clara no terminal, o que garante o retorno seguro do utilizador ao menu em que se encontrava.

```

try {
    Data viagem(diaC, mesC, anoC, diaS, mesS, anoS);

    std::string opcao;
    while (true) {
        std::cout << "Deseja adicionar um horario? (s/n):" << std::endl;
        std::getline(std::cin, opcao);
        if (opcao != "s" && opcao != "S") break;

        std::string horario;
        std::cout << "Introduza o horario e atividade:" << std::endl;
        std::getline(std::cin, horario);
        viagem.addHorario(horario);
    }

    std::cout << "\n--- DADOS DA VIAGEM ---" << std::endl;
    Calendario ch = viagem.getChegada();
    Calendario sa = viagem.getSaida();

    std::cout << "Data de Chegada: " << ch.dia << "/" << ch.mes << "/" << ch.ano <<
std::endl;
    std::cout << "Data de Saida: " << sa.dia << "/" << sa.mes << "/" << sa.ano << std::endl;
    std::cout << "Duracao: " << viagem.getNumeroDias() << " dias" << std::endl;

    std::cout << "Horarios planeados:" << std::endl;
    for (const std::string& h : viagem.getHorarios()) {
        std::cout << " -> " << h << std::endl;
    }

} catch (const DataInvalidaException& e) {
    std::cerr << e.what() << std::endl;
}

```

10. Conclusão

Em suma, o desenvolvimento do Sistema de Gestão de Planos de Viagem (SGPV) cumpriu integralmente os objetivos propostos pela unidade curricular, resultando numa aplicação robusta, funcional e, acima de tudo, sustentada pelos mais rigorosos padrões de Engenharia de Software.

O projeto provou que o domínio da Programação Orientada a Objetos em C++ vai muito além da simples escrita de código que "funciona". Através da adoção da arquitetura MVC (Model-View-Controller), foi possível garantir uma separação clara de responsabilidades: a lógica de negócio opera de forma independente, as interfaces gerem exclusivamente a interação com o utilizador e os controladores orquestram o fluxo de forma harmoniosa.

Do ponto de vista técnico, a qualidade do sistema reflete-se na aplicação prática e exaustiva dos princípios SOLID e padrões GRASP. A utilização de herança e Polimorfismo (permitindo o tratamento unificado de Países, Cidades e Atividades no roteiro) evidencia o respeito pelo Princípio da Substituição de Liskov e pela Inversão de Dependência. O sistema garante Alta Coesão e Baixo Acoplamento ao isolar domínios de conhecimento — delegando, por exemplo, o cálculo de impostos a uma "Fabricação Pura" (a classe `Financeiro`) e a validação de calendários ao seu "Especialista de Informação" (a classe `DataHora`).

Adicionalmente, a resiliência da aplicação foi assegurada através de uma hierarquia customizada de Tratamento de Exceções, que previne falhas críticas em tempo de execução, enquanto a rigorosa padronização de nomenclatura (*UpperCamelCase* para tipos e *lowerCamelCase* para variáveis e métodos) garante a total legibilidade e manutenibilidade do código por futuros programadores.

Conclui-se, assim, que o SGPV não é apenas um simulador logístico de viagens, mas sim a materialização de um software escalável, seguro e extensível, atestando a profunda compreensão das metodologias e paradigmas de desenvolvimento de software exigidos num contexto profissional.